# URDF-GUI für Bahnplanung

## Entwickler- und Bedienerdokumentation

Stand: urdf_gui.py 2.3 · Oktober 2026

# Inhalt


## Abkürzungen und Fachbegriffe

Abkürzungen
Fachbegriffe

## Teil I · Entwicklerdokumentation

1 Zweck und Umfang
2 Systemumgebung und Start
3 Architektur
4 Module im Einzelnen
5 Schnittstellen
6 Abläufe
7 Erweitern und Ändern
8 Tests
9 Bekannte Einschränkungen und offene Punkte
Anhang A · Konfigurationsreferenz

## Teil II · Bedienerdokumentation

1 Überblick
2 Installation
3 Starten und Beenden
4 Hauptfenster
5 Fenster „Bewegungsparameter“
6 Fenster „Fahren“
7 Fenster „Punktfolge“
8 Fenster „URDF bearbeiten“
9 Bahnansicht (`bahn_viewer3.py`)
10 Sperren und Sicherheit im Programm
11 Meldungen und Fehlerbehebung
12 Dateien und Ablageorte
13 Fehlende Funktionen und Ausblick

# Abkürzungen und Fachbegriffe

> Gilt für beide Teile. Fachbegriffe aus Robotik, ROS 2, MoveIt und Softwaretechnik stehen im Text so, wie sie dort heißen, also meist englisch (Node, Service, Callback, Watchdog, Inverse Kinematics …). Wo der Code oder die Oberfläche einen deutschen Namen verwendet, steht er in der mittleren Spalte; in der Anleitung werden Beschriftungen der Oberfläche immer wörtlich zitiert.

## Abkürzungen

| Abkürzung | ausgeschrieben | Bedeutung hier |
|---|---|---|
| A, B, C | – | Winkel der TCP-Orientierung in Grad: erste, zweite, dritte Drehung der gewählten Drehkonvention |
| ACM | Allowed Collision Matrix | Linkpaare, die nicht auf Kollision geprüft werden (`disable_collisions` in der SRDF) |
| CIRC | Circular Motion | Kreisbewegung des Pilz-Planners; braucht einen Hilfspunkt |
| CSV | Comma-Separated Values | Textformat der Bahnen (`TCP_Fahrten/`), hier mit `;` getrennt |
| FK | Forward Kinematics | Lage aller Links aus den Achswerten |
| GUI | Graphical User Interface | die Oberfläche `urdf_gui.py` |
| IK | Inverse Kinematics | Achswerte zu einer vorgegebenen TCP-Lage; kommt immer aus MoveIt |
| IPO | Interpolation | Interpolationstakt der GUI (`IPO_TICK_MS` = 20 ms) |
| JSON | JavaScript Object Notation | Format von Auftrag und Antwort der Kindprozesse und des Wrappers |
| KDL | Kinematics and Dynamics Library (Orocos) | Standard-IK-Solver von MoveIt |
| LIN | Linear Motion | TCP fährt auf einer Geraden im kartesischen Raum |
| LMA | Levenberg-Marquardt-Algorithmus | alternativer IK-Solver von MoveIt |
| PID | Process Identifier | Prozessnummer, Teil des Node-Namens `move_group_gui<PID>_<n>` |
| PTP | Point-to-Point | alle Achsen fahren synchron ins Ziel; die TCP-Bahn ist nicht vorgegeben |
| QoS | Quality of Service | Zustellregeln eines ROS-Topics (z. B. `transient local` für `robot_description`) |
| rclcpp, rclpy | ROS Client Library für C++ bzw. Python | Programmierschnittstelle zu ROS 2 |
| ROS 2 | Robot Operating System 2 | Middleware; verwendet wird die Distribution Jazzy |
| rpy | Roll, Pitch, Yaw | Winkel in der URDF (`<origin rpy="…">`), immer ZYX |
| RViz2 | ROS Visualization | 3D-Anzeige des Roboters |
| SRDF | Semantic Robot Description Format | Planning Group, Group States, ACM; wird von der GUI erzeugt |

| Abkürzung | ausgeschrieben | Bedeutung hier |
|---|---|---|
| TCP | Tool Center Point | Werkzeugpunkt; hier der Tip-Link bzw. der gewählte Werkzeug-Frame |
| TF | Transform (Paket `tf2`) | Lagen aller Frames zueinander, Topic `/tf` |
| TOPP-RA | Time-Optimal Path Parameterization based on Reachability Analysis | Time Parameterization aus Intrinsic Core |
| TOTG | Time-Optimal Trajectory Generation | Time Parameterization aus MoveIt |
| TRAC-IK | – | alternativer IK-Solver (eigenes Plugin) |
| URDF | Unified Robot Description Format | XML-Modell des Roboters: Links, Joints, Grenzen, Aussehen |
| WSL, WSLg | Windows Subsystem for Linux (mit GUI-Unterstützung) | Ubuntu unter Windows; WSLg zeigt die Linux-Fenster an |
| YAML | YAML Ain't Markup Language | Format der Parameterdatei von `move_group` |
| ZYX, ZYZ, ZXZ | – | Drehkonventionen für A, B, C (intrinsisch), siehe Teil I, Abschnitt 4.2 |

# Fachbegriffe

| Begriff | im Code / in der Oberfläche | Bedeutung |
|---|---|---|
| Achsziel | `achsziel`, Zeile `J …` | Ziel in Achswerten statt als Lage; fährt immer PTP mit Radius 0 |
| Auftrag, Antwort | `auftrag.json`, `antwort.json`, `request_*` | Daten an `RosBridge` oder einen Kindprozess und deren Ergebnis |
| Blending | Überschleifen, `blend_radius`, `radius=` | Übergang in den nächsten Punkt ohne Halt; Radius 0 heißt Genauhalt |
| Callback | – | Funktion, die bei einem Ereignis aufgerufen wird (Timer, Umschalten der Drehkonvention) |
| Capability | `capabilities` | Erweiterung von `move_group`, hier `MoveGroupSequenceService` für Punktfolgen |
| Cartesian Limits | Reiter „Kartesisch", `cartesian_limits` | Bahngrenzen für LIN und CIRC bei Pilz |
| Endeffektor | „Mehrere Endeffektoren …" | Ende der kinematischen Kette, an dem das Werkzeug sitzt |
| Exception | – | Ausnahmefehler in Python; die Takte fangen sie ab und melden sie |
| Executor | `SingleThreadedExecutor` | arbeitet Timer und Callbacks eines ROS-Nodes ab; hier im eigenen Thread |
| Frame | – | Koordinatensystem; jeder Link hat eines |
| Genauhalt | Radius 0 | der Roboter hält im Punkt an |
| Glied | „Glied anhaengen", `append_limb` | Joint mit dem dahinterliegenden Link |
| Group State | `group_state` „null" | benannte Achsstellung in der SRDF |
| Halte-Taster | „FAHREN (HALTEN)", `_hold_*` | plant und fährt, solange er gedrückt ist; Loslassen bremst |
| Hilfspunkt | `hilfspunkt` | Zwischenpunkt, durch den eine CIRC-Bewegung läuft |
| IK-Ast | `ast_folgen` | eine von mehreren Achslösungen zur selben Lage (Roboterkonfiguration) |

| Begriff | im Code / in der Oberfläche | Bedeutung |
|---|---|---|
| Jerk | Ruck, `jerk` | dritte Ableitung der Lage nach der Zeit |
| Joggen | `_jog_*` | Verfahren in kleinen Schritten von Hand |
| Joint | Gelenk | verbindet zwei Links: `revolute`, `continuous`, `prismatic`, `fixed` |
| Joint Limits | Reiter „Achsgrenzen", `joint_limits` | Grenzen für Geschwindigkeit, Beschleunigung, Verzögerung und Jerk je Achse |
| Joint Space | Achsraum, Gelenkraum | Raum der Achswerte, im Gegensatz zum kartesischen Raum |
| Kindprozess | `TotgProzess`, `MoveItProcess` | eigener Prozess, den die GUI startet und beendet |
| Kopplung | `<mimic>`, „KOPPLUNG" | Achse folgt einer anderen: Faktor · Führung + Versatz |
| Lage | „TCP-LAGE", `lage` | Pose: Position (X, Y, Z) und Orientierung (A, B, C) |
| Leaf-Link | Blatt, `leaf_links` | Link ohne Nachfolger; Kandidat für den TCP |
| Link | – | starrer Körper des Modells |
| Lock | `threading.Lock` | Sperre für Daten, die zwei Threads teilen |
| Mock, Stub | – | Attrappe im Test, die ein echtes Modul oder Programm ersetzt |
| Modalzustand | `modal`, `MODAL_START` | in der Punktliste gesetzte Werte gelten für alle folgenden Zeilen |
| `move_group` | – | Node von MoveIt für Planung und IK; hier ein Kindprozess der GUI |
| MoveItPy | `moveit_py` | Python-Schnittstelle zu MoveIt mit eigenem rclcpp-Node |
| Node | – | Teilnehmer im ROS-Graphen, z. B. `urdf_gui_tf` |
| Nutzlast | `Folge.nutzlast` | hier: Payload, also die Daten einer Anfrage – nicht die Traglast des Roboters |
| Override | „Override Geschwindigkeit", `velocity_scaling` | Prozentfaktor auf Geschwindigkeit bzw. Beschleunigung |
| Pilz | `pilz_industrial_motion_planner` | Planner von MoveIt für PTP, LIN, CIRC und Punktfolgen |
| Planner, Planning Pipeline | `planning_pipelines` | Planungsverfahren von MoveIt und die Kette aus Planner und Adaptern |
| Planning Group | Planungsgruppe, `manipulator` | Satz von Joints, für den geplant wird; hier eine Kette von der Wurzel zum Tip |
| Planning Scene Monitor | – | hält in MoveIt das aktuelle Modell und die Umgebung |
| Punktfolge, Punktliste | `sequenz`, Fenster „Punktfolge" | Folge von Zielen, die als eine Bahn geplant wird |
| Request/Response Adapter | `request_adapters`, `response_adapters` | Prüf- und Nachbearbeitungsschritte um den Planner |
| Resampling | Neuabtastung, `lin_neuabtastung`, `resample_dt` | neue, gleichmäßige Abtastung einer Bahn |
| Root-Link | Wurzel, `root_links` | erster Link des Modells; Bezug aller Lagen |
| Ruckig | „Ruckig-Glaettung" | Bibliothek, die eine Bahn auf begrenzten Jerk glättet |
| Service | Dienst | Anfrage/Antwort im ROS-Graphen, z. B. `/compute_ik` |

| Begriff | im Code / in der Oberfläche | Bedeutung |
|---|---|---|
| Session | – | Prozessgruppe eines Kindprozesses; wird als Ganzes beendet |
| Singularität | – | Achsstellung, in der der Roboter einen Freiheitsgrad verliert; davon zu trennen ist die Winkelsingularität der Darstellung A, B, C |
| Slider | Schieber, `_on_slider` | Schieberegler je Achse im Hauptfenster |
| Stand | `STAND`, `MODUL_STAENDE` | Versionsnummer einer Datei bzw. einer Schnittstelle |
| State Machine | Ablaufsteuerung, `STATE_*`, `_set_state` | Zustände `BEREIT`, `IK`, `PLANEN`, `FOLGE IK`, `FOLGE`, `FAHREN`, `BREMSEN`, `FEHLER` |
| Stützpunkt | `points`, `punkte` | Punkt einer Trajektorie: Zeit, Achswerte, Geschwindigkeiten |
| Tag | Auftragsnummer, `tag` | Nummer, an der die GUI die Antwort zu ihrem Auftrag erkennt |
| Teilbahn, Abschnitt | `teilbahnen`, `abschnitte` | Stück einer Folge zwischen zwei Halten |
| Thread | – | Ausführungsstrang innerhalb eines Prozesses |
| Time Parameterization | Zeitparametrierung | ordnet einem geometrischen Pfad einen Zeitverlauf zu (TOTG, TOPP-RA) |
| Timeout | Frist, `timeout`, `FRIST_S` | Zeit, nach der ein Auftrag als gescheitert gilt |
| Timer | – | ruft einen Callback in festem Takt auf |
| Tip-Link | `tip` | letzter Link der Planning Group; an ihm rechnen IK und LIN |
| Topic | – | Datenkanal im ROS-Graphen, z. B. `/joint_states` |
| Trajektorie, Bahn | `TrajectoryPlayer` | Weg mit Zeitverlauf |
| Umkehrpunkt | `umkehr` | Punkt, an dem eine Achse die Richtung wechselt; dort wird angehalten |
| Verrundung | `rundung_translation_m`, `rundung_rot` | kartesische Eckenrundung bei Intrinsic |
| Wall-Clock-Zeit | Wanduhr, `wall_drift` | tatsächlich vergangene Zeit, im Gegensatz zur Bahnzeit aus den Takten |
| Watchdog | `_hold_watch`, `_moveit_watch`, `_tf_watchdog` | prüft zyklisch einen Zustand und greift ein, wenn er nicht stimmt |
| Wrapper | `intrinsic_toppra_wrapper` | eigenes C++-Programm, das Funktionen von Intrinsic Core über JSON aufrufbar macht |
| Zielraum | „Achsraum“, „Kartesisch (IK)“ | Art, in der im Fahrfenster das Ziel vorgegeben wird |

# Teil I · Entwicklerdokumentation

> Für Entwicklerinnen und Entwickler, die am Code weiterarbeiten. Vorausgesetzt werden Python, Grundkenntnisse in ROS 2 und MoveIt 2 sowie Robotik-Grundbegriffe (Forward und Inverse Kinematics, Time Parameterization). Alle Angaben beziehen sich auf den Stand im Repository vom Oktober 2026 (`urdf_gui.py` Fassung 2.3). Abkürzungen und Fachbegriffe erklärt das Verzeichnis vor diesem Teil; die Installation beschreibt Teil II, Abschnitt 2.

## 1 Zweck und Umfang

Die Software ist eine Tkinter-Oberfläche für ROS 2 (Jazzy) und MoveIt 2. Sie

- lädt eine URDF, zeigt Gelenke, Links und Frames an und lässt das Modell bearbeiten (Glieder anhängen, einfügen, herausnehmen, Kopplungen, Aussehen),
- rechnet die Forward Kinematics (FK) selbst und sendet alle Frames auf `/tf`, dazu `robot_description` und `/joint_states` (Anzeige in RViz2),
- startet `move_group` als eigenen Kindprozess mit einer selbst erzeugten Parameterdatei (URDF, SRDF, Joint Limits, IK-Solver, Pilz-Pipeline),
- plant Einzelfahrten (PTP/LIN mit Pilz) und Punktfolgen mit drei vergleichbaren Verfahren – **Pilz**, **TOTG** und **Intrinsic** –,
- tastet die geplanten Bahnen selbst im 20-ms-Takt ab („fährt“ sie also nur virtuell über `/tf` und `/joint_states`; es gibt keinen Controller),
- zeichnet die TCP-Bahn mit 25 Hz auf und schreibt sie als CSV.

Ein zweites, eigenständiges Programm, `bahn_viewer3.py`, zeigt aufgezeichnete und geplante Bahnen in 3D an und vergleicht mehrere Bahnen.

Kommentare, Docstrings, Meldungen und Beschriftungen sind deutsch (ohne Umlaute); die Bezeichner sind teils deutsch, teils englisch (Abschnitt 1.2). Docstrings erklären bewusst das *Warum* einer Entscheidung; bei Änderungen lohnt es sich, sie zu lesen, bevor man eine scheinbar überflüssige Prüfung entfernt.

### 1.1 Stände der Dateien

Jede Datei, die eine Schnittstelle zu einer anderen hat, trägt eine Stand-Nummer. Die Oberfläche prüft sie beim Start bzw. beim Auftrag.

| Datei | Konstante | Wert | geprüft durch |
|---|---|---|---|
| `einstellungen`, `moveit_konfig`, `totg_prozess`, `ros_bruecke` | `STAND` | 2 | `urdf_gui.MODUL_STAENDE` / `pruefe_stand()` beim Import, Abbruch bei Abweichung |
| `stil`, `matrix`, `anzeige`, `urdf_modell`, `kinematik`, `bahn` | `STAND` | 1 | ebenso |
| `totg_folge.py` | `STAND` | 2 | `urdf_gui.py` verlangt ≥ 2; geht als `stand` in den TOTG-Auftrag |
| `totg_rechner.py` | `STAND` | 2 | lehnt Aufträge mit höherem `stand` ab |

| Datei | Konstante | Wert | geprüft durch |
|---|---|---|---|
| `intrinsic_folge.py` | `STAND` / `AUFTRAG_STAND` | 1 / 6 | `urdf_gui.py` verlangt genau 1; `AUFTRAG_STAND` geht in den Auftrag |
| `intrinsic_lin_rechner.py` | `STAND` / `WRAPPER_STAND` | 6 / 3 | lehnt Aufträge mit höherem Stand ab; verlangt Wrapper ≥ 3 |
| `intrinsic_lin_auftrag.py` | `STAND` | 1 | – (Hilfsskript) |

## 1.2 Namen im Code: Deutsch und Englisch

Der Code mischt beide Sprachen, auch innerhalb einer Datei und manchmal innerhalb eines Namens (`_seq_achsziel_uebernehmen`). Das ist mit den Fassungen gewachsen: Die älteren Kernteile (Modell, Kinematik, Fahren, ROS, MoveIt) sind englisch benannt, die später hinzugekommenen Teile (Drehkonvention, Punktfolge, TOTG, Intrinsic, Kindprozesse) deutsch. Es gibt kein Modul, in dem eine Sprache die andere vollständig ersetzt hätte. Das Muster:

| Bereich | Sprache | Beispiele |
|---|---|---|
| Kommentare, Docstrings, Meldungen, Beschriftungen | Deutsch, ohne Umlaute | „Uebernehmen“, „Plan verworfen: …“ |
| Dateinamen | Deutsch | `einstellungen.py`, `ros_bruecke.py`, `sequenz.py` |
| ältere Kernteile | Englisch | `UrdfViewer`, `RosBridge`, `MoveItConfig`, `TrajectoryPlayer`, `PathRecorder`, `forward_kinematics`, `append_limb`, `_drive_plan` |
| neuere Teile | Deutsch | `Folge`, `Punkt`, `Befund`, `lies_liste`, `anfrage`, `antwort_lesen`, `TotgProzess.starten`, `Anzeigekonvention.zerlege`, `MoveItKinematik` |
| Namen, die ROS oder MoveIt vorgeben | Englisch, unverändert | `velocity_scaling`, `planner_id`, `blend_radius`, `max_trans_vel`, `path_tolerance`, `resample_dt` |
| Nutzlast an `RosBridge` | Englisch (Felder der ROS-Nachrichten) | `group`, `goal`, `start`, `tolerance` |
| JSON-Schlüssel der Kindprozesse und des Wrappers | Deutsch, außer MoveIt-Namen | `wegpunkte`, `abschnitte`, `umkehr`, `posen`, aber `velocity_scaling` |
| Konstanten | überwiegend Englisch | `IPO_TICK_MS`, `TF_RATE_HZ`; deutsch u. a. `FRIST_S`, `FOLGE_START_TOL`, `MODUL_STAENDE` |
| Zustände | englischer Name, deutscher Wert | `STATE_READY = "BEREIT"`, `STATE_MOVING = "FAHREN"` |
| Methoden von `UrdfViewer` | Englisch; im Bereich Punktfolge auch Deutsch oder gemischt | `_seq_read`, `_totg_send`, `_seq_herkunft_bauen`, `_tick_rumpf` |
| Kopfzeilen der CSV | Deutsch | `#roboter`, `#drehkonvention`, `#verfahren` |

Für Erweiterungen gilt: der Sprache des umgebenden Moduls folgen und Namen, die ROS oder MoveIt vorgeben, nie übersetzen. Häufige deutsche Namen und ihre englische Entsprechung:

| Name im Code | englisch | Bedeutung |
|---|---|---|
| `auftrag`, `antwort` | request, response | Daten an einen Kindprozess und sein Ergebnis |
| `Folge`, `Punkt`, `Schritt` | sequence, point, step | Punktfolge, Zeile der Liste, aufgelöste Zeile |

| Name im Code | englisch | Bedeutung |
|---|---|---|
| `Befund` (`fehler`, `warnung`, `hinweis`) | finding | Ergebnis einer Prüfung |
| `nutzlast` | payload | Daten der Anfrage an `/plan_sequence_path` |
| `lage`, `achsziel` | pose, joint goal | Ziel als Lage bzw. als Achswerte |
| `fahrart`, `tempo`, `beschleunigung`, `radius` | motion type, velocity/acceleration scaling, blend radius | Felder der Punktliste |
| `wegpunkte`, `abschnitte`, `teilbahnen`, `umkehr` | waypoints, segments, sub-trajectories, reversal | Aufbau einer Folge |
| `grenzen`, `frist`, `stand` | limits, timeout, version | |
| `herkunft` | provenance | Verfahren und Parameter, die in die CSV geschrieben werden |
| `rechner` | solver | Kindprozess-Skript (`totg_rechner.py`, `intrinsic_lin_rechner.py`) |
| `verschleifen`, `ueberschleifen`, `rundung` | blend, corner rounding | Übergänge an den Ecken |
| `abtastung`, `neuabtastung` | sampling, resampling | |
| `kugel` (`kugeln_pruefen`) | blend sphere | Überschleifkugel um einen Punkt |
| `ecke`, `eckwerte` | corner, corner metrics | Abstand und Geschwindigkeit an einem Zwischenpunkt |
| `ast_folgen` | follow IK branch | Weiterrechnen auf einem anderen IK-Ast |
| `lies_…`, `pruefe_…`, `baue`, `zerlege` | read, check, build, decompose | Verben am Namensanfang |

Eine Vereinheitlichung wäre eine reine Umbenennung, berührt aber die JSON-Schlüssel (neuer `AUFTRAG_STAND`), die Tests, die `urdf_gui.py` als Quelltext lesen, und die Kopfzeilen bestehender CSV-Dateien. Sie steht deshalb als offener Punkt im Ausblick (Teil II, Abschnitt 13).

# 2 Systemumgebung und Start

## 2.1 Voraussetzungen

Die Installation Schritt für Schritt steht in Teil II, Abschnitt 2.

| Komponente | Zweck | Hinweis |
|---|---|---|
| ROS 2 Jazzy, `rclpy`, `tf2_ros` | TF, Topics, Services | ohne ROS startet die GUI trotzdem („Nur Anzeige und FK“) |
| MoveIt 2 mit Pilz-Planner (`moveit_ros_move_group`) | Planung, IK | getestet mit `moveit_core` 2.12.4 |
| `moveit_py` (`ros-jazzy-moveit-py`) | TOTG- und Intrinsic-Rechner | `python3 totg_rechner.py --pruefen` |
| RViz2 | Anzeige | optional (`rviz:=false`) |
| Python 3.12 mit `tkinter`, `PyYAML` | GUI, Parameterdateien | NumPy nur im Intrinsic-Rechner (Singulärwerte) |

| Komponente | Zweck | Hinweis |
|---|---|---|
| `intrinsic_toppra_wrapper` | Kurve, Resampling, TOPP-RA aus Intrinsic Core | Bazel-Build, Pfad über `INTRINSIC_WRAPPER` |
| Pillow (optional) | JPEG/PNG-Export im Bahn-Viewer | sonst PostScript (`.eps`) |

## 2.2 Starten

```
source /opt/ros/jazzy/setup.bash
cd ~/Display
ros2 launch display_launch4.py                    # GUI + RViz2
ros2 launch display_launch4.py urdf:=/pfad/robot.urdf
ros2 launch display_launch4.py rviz:=false software_gl:=false
python3 urdf_gui.py [URDF]                        # ohne Launch
python3 bahn_viewer3.py [CSV ...]                 # ohne ROS
```

`display_launch4.py` wird nicht als Package installiert; es löst alle Pfade relativ zu sich selbst auf und muss neben `urdf_gui.py` liegen.

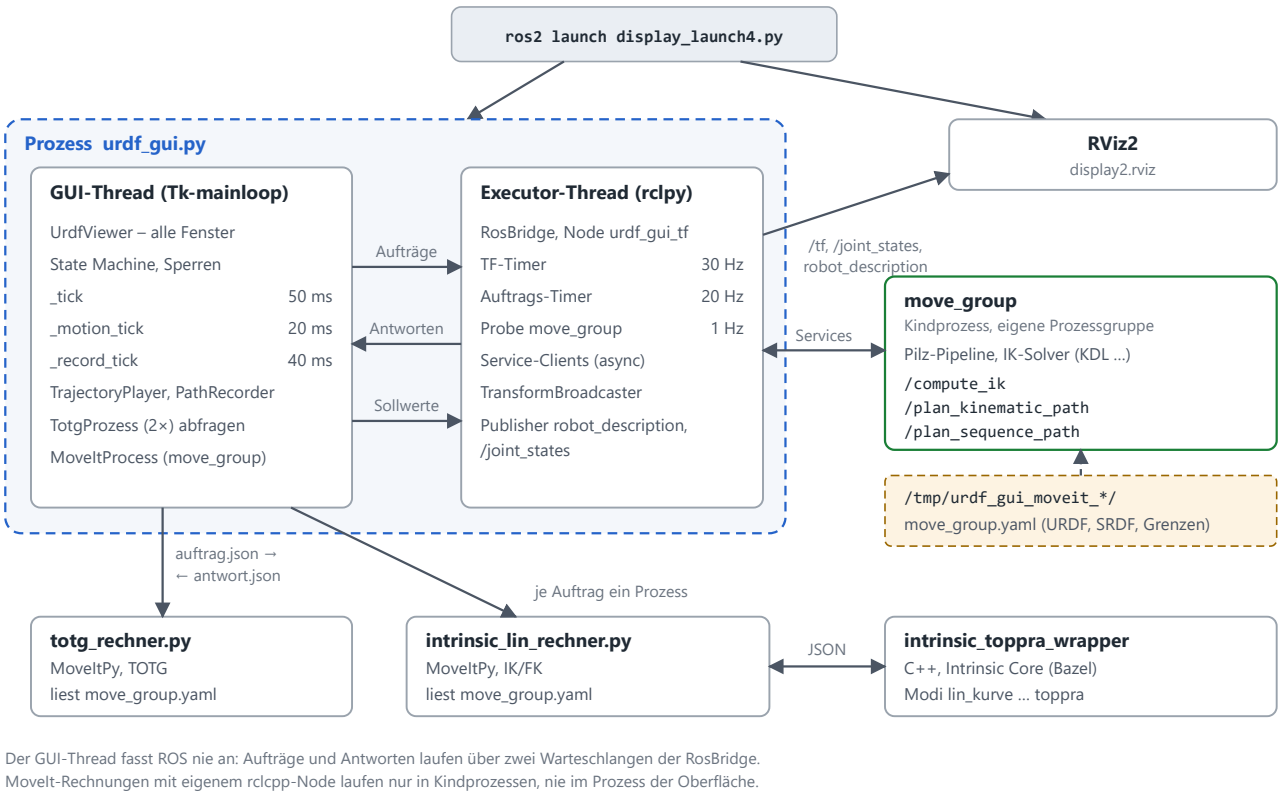
| Launch-Argument | Standard | Wirkung |
|---|---|---|
| `urdf` | `bmKinematic.urdf` neben dem Skript | geladenes Modell (Argument an `urdf_gui.py`) |
| `rviz` | `true` | RViz2 mit 1,5 s Verzögerung mitstarten |
| `rviz_config` | `display2.rviz` | RViz-Konfiguration |
| `software_gl` | `true` | setzt `LIBGL_ALWAYS_SOFTWARE=1` (unter WSL nötig) |
| `intrinsic_wrapper` | `$INTRINSIC_WRAPPER` oder `~/intrinsic-core/bazel-bin/intrinsic_toppra_wrapper/intrinsic_toppra_wrapper` | wird als Umgebungsvariable `INTRINSIC_WRAPPER` an die GUI gereicht |

Die GUI läuft als `ExecuteProcess` mit `on_exit=Shutdown`: Schließt man das Fenster, endet das gesamte Launch. Ohne URDF-Argument gilt `URDF_PATH` aus `einstellungen.py`, also `bmKinematic.urdf` neben den Skripten.

| Umgebungsvariable | Wirkung |
|---|---|
| `INTRINSIC_WRAPPER` | Pfad zum Intrinsic-Wrapper (setzt auch das Launch-Argument) |
| `URDF_GUI_URDF` | URDF ohne Startargument |
| `URDF_GUI_SICHERUNG` | Ablage für Sicherungen der URDF |
| `URDF_GUI_BAHNEN` | Ablage für CSV-Bahnen |
| `LIBGL_ALWAYS_SOFTWARE` | Software-Rendering für RViz (über `software_gl`) |

# 3 Architektur

## 3.1 Prozesse und Threads



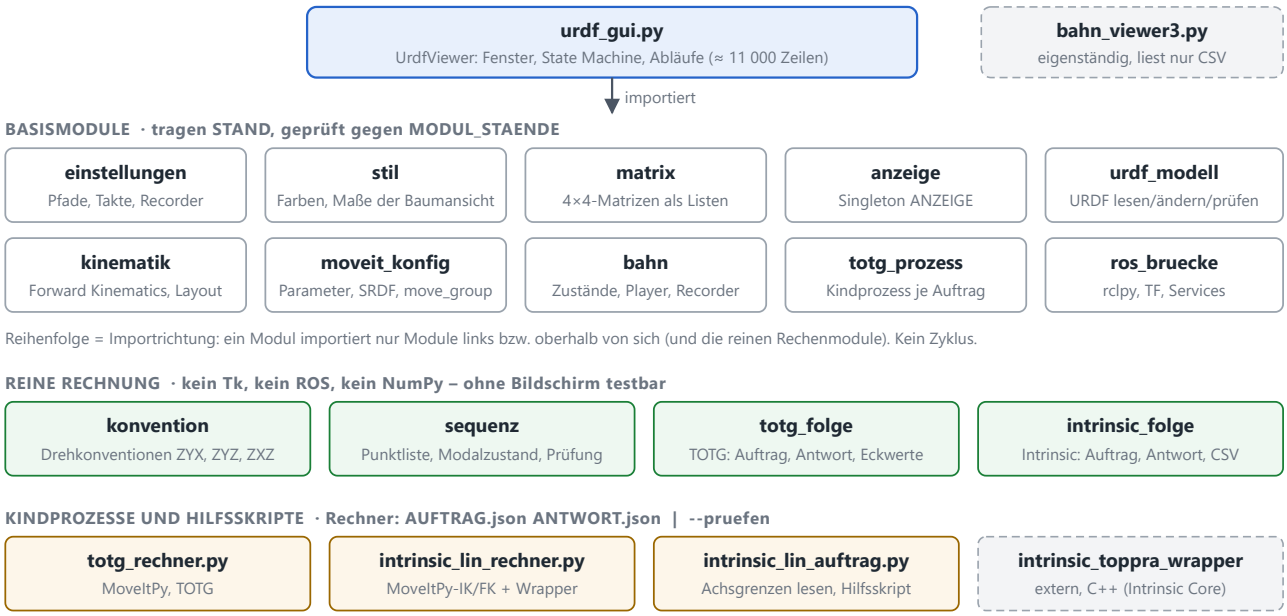

Der GUI-Thread fasst ROS nie an: Aufträge und Antworten laufen über zwei Warteschlangen der RosBridge. MoveIt-Rechnungen mit eigenem rclcpp-Node laufen nur in Kindprozessen, nie im Prozess der Oberfläche.

*Prozesse und Threads zur Laufzeit*

Zur Laufzeit gibt es bis zu fünf Arten von Prozessen:

1. **`urdf_gui.py`** – ein Prozess mit zwei Threads:
   - *GUI-Thread* (Tk-`mainloop`): alle Fenster, die State Machine, die Interpolation der Bahn, der Recorder. Er fasst ROS **nie** an.
   - *Executor-Thread* der `RosBridge`: ein `SingleThreadedExecutor` mit drei Timern (TF 30 Hz, Aufträge 20 Hz, Probe 1 Hz) in einer eigenen `spin_once`-Schleife, die Exceptions fängt, statt zu sterben.
2. **`move_group`** – gestartet von `MoveItProcess` als Kindprozess in eigener Prozessgruppe, mit eindeutigem Node-Namen `move_group_gui<PID>_<n>`.
3. **`totg_rechner.py` / `intrinsic_lin_rechner.py`** – je Auftrag ein kurzer Kindprozess (über `TotgProzess`), weil `moveit_py` einen eigenen rclcpp-Node mitbringt, der nicht in den GUI-Prozess gehört.
4. **`intrinsic_toppra_wrapper`** – C++-Programm aus Intrinsic Core, je Rechenschritt einmal vom Intrinsic-Rechner aufgerufen.
5. **RViz2** – reiner Anzeiger.

## 3.2 Schichten und Module




*Module und ihre Abhängigkeiten*

| Datei | Verantwortung | wichtigste Namen |
|---|---|---|
| `urdf_gui.py` | Oberfläche, Ablaufsteuerung, alle Fenster | `UrdfViewer`, `MODUL_STAENDE`, `pruefe_stand()`, `main()` |
| `einstellungen.py` | Pfade, Takte, Grenzen des Recorders | `HIER`, `URDF_PATH`, `BACKUP_DIR`, `BAHN_DIR`, `BAHN_VIEWER`, `TF_RATE_HZ`, `RECORD_*` |
| `stil.py` | Farbpalette, Maße der Baumansicht | `BG`, `ACCENT`, `COLOR_*`, `TREE_*` |
| `matrix.py` | 4×4-Matrizen als Listen (ohne NumPy) | `multiply`, `invert`, `from_axis_angle`, `to_quaternion` |
| `konvention.py` | Drehkonventionen ZYX, ZYZ, ZXZ (intrinsisch) | `from_abc_xyz`, `to_xyz_abc`, `from_rpy_xyz`, `to_xyz_rpy`, `ist_singulaer` |
| `anzeige.py` | gewählte Anzeigekonvention, Singleton | `Anzeigekonvention`, `ANZEIGE` |
| `urdf_modell.py` | URDF lesen, ändern, prüfen, speichern | `parse`, `validate_root`, `append_limb`, `insert_limb`, `remove_limb`, `save_urdf` |
| `kinematik.py` | Forward Kinematics, Layout des Gelenkbaums | `forward_kinematics`, `root_links`, `leaf_links`, `tree_layout` |
| `moveit_konfig.py` | Konfiguration, SRDF, Parameterdatei, `move_group`-Prozess | `MoveItConfig`, `MoveItProcess`, `build_srdf` |
| `bahn.py` | Zustände, Interpolation, Abtaster, Recorder | `TrajectoryPlayer`, `PathRecorder`, `hermite`, `STATE_*` |
| `totg_prozess.py` | Rechner als kurzer Kindprozess | `TotgProzess` |
| `ros_bruecke.py` | rclpy-Node, TF, Service-Aufrufe | `RosBridge` |
| `sequenz.py` | Punktliste, Modalzustand, Prüfung, Nutzlast | `lies_liste`, `Folge`, `Punkt`, `Befund`, `loese_achsziel` |
| `totg_folge.py` | TOTG-Auftrag, Antwort, Eckwerte, CSV-Kopf | `anfrage`, `antwort_lesen`, `abschnitte`, `eckwerte` |

| Datei | Verantwortung | wichtigste Namen |
|---|---|---|
| `intrinsic_folge.py` | Intrinsic-Auftrag, Antwort, CSV-Kopf | `anfrage`, `antwort_lesen`, `pruefe_parameter` |
| `totg_rechner.py` | TOTG über `moveit_py` (Kindprozess) | `rechne`, `params_fuer_moveitpy`, `verketten` |
| `intrinsic_lin_rechner.py` | LIN/PTP nach Intrinsic Core mit MoveIt-IK (Kindprozess) | `rechne`, `MoveItKinematik`, `ik_kette`, `tcp_pruefung` |
| `intrinsic_lin_auftrag.py` | Joint Limits aus der Parameterdatei; Hilfsskript TOTG→LIN-Auftrag | `parameter_lesen`, `grenzen_lesen` |
| `bahn_viewer3.py` | 3D-Bahnansicht, eigenständig | `BahnViewer`, `Track`, `Camera` |

Abhängigkeiten laufen nur in eine Richtung: Die Basismodule bilden eine Kette `einstellungen → stil → matrix → anzeige → urdf_modell → kinematik → moveit_konfig → bahn → totg_prozess → ros_bruecke`, in der jedes Modul nur Vorgänger und die reinen Rechenmodule importiert. `urdf_gui.py` importiert alles, importiert wird es von niemandem (die Tests lesen es als Text, siehe Abschnitt 8).

## 3.3 Leitlinien im Code

Diese Regeln ziehen sich durch alle Module. Wer sie bricht, bekommt Fehler, die schwer zu finden sind.

**Ein Thread fasst ROS an.** Der GUI-Thread baut keine ROS-Nachrichten und wartet nie auf eine Antwort. Er legt Aufträge als reine Python-Daten (`dict` aus Zahlen und Zeichenketten) in `RosBridge._jobs` ab; der Executor-Thread baut daraus die Nachricht und legt die Antwort, wieder als reine Daten, in `RosBridge.results` ab. Abgeholt wird im Fußzeilentakt (`_tick`, 50 ms). Einzige Ausnahme ist `set_positions()`, das unter einem Lock die Sollwerte übergibt.

**Jede Antwort trägt einen Tag.** `_job_open()` vergibt die Auftragsnummer `tag`, `_job_mine()` lässt nur die erwartete Antwort durch, `_job_drop()` verwirft alles Ausstehende. So kann eine verspätete Antwort eines abgebrochenen Auftrags nie als neue gelten.

**MoveIt-Rechnungen mit eigenem Node laufen im Kindprozess.** `moveit_py` bringt eine eigene rclcpp-Laufzeit mit. Deshalb laufen TOTG und die Intrinsic-IK in `totg_rechner.py` bzw. `intrinsic_lin_rechner.py`, nie im GUI-Prozess.

**Eine Kinematik für die Planung.** Die Inverse Kinematics (IK) kommt ausschließlich aus MoveIt (`/compute_ik` bzw. `moveit_py` mit derselben Parameterdatei wie `move_group`). Die GUI rechnet die Forward Kinematics (FK) für Anzeige, TF und Aufzeichnung selbst aus der URDF (`kinematik.forward_kinematics`); die Rechner nutzen für ihre FK wieder MoveIt.

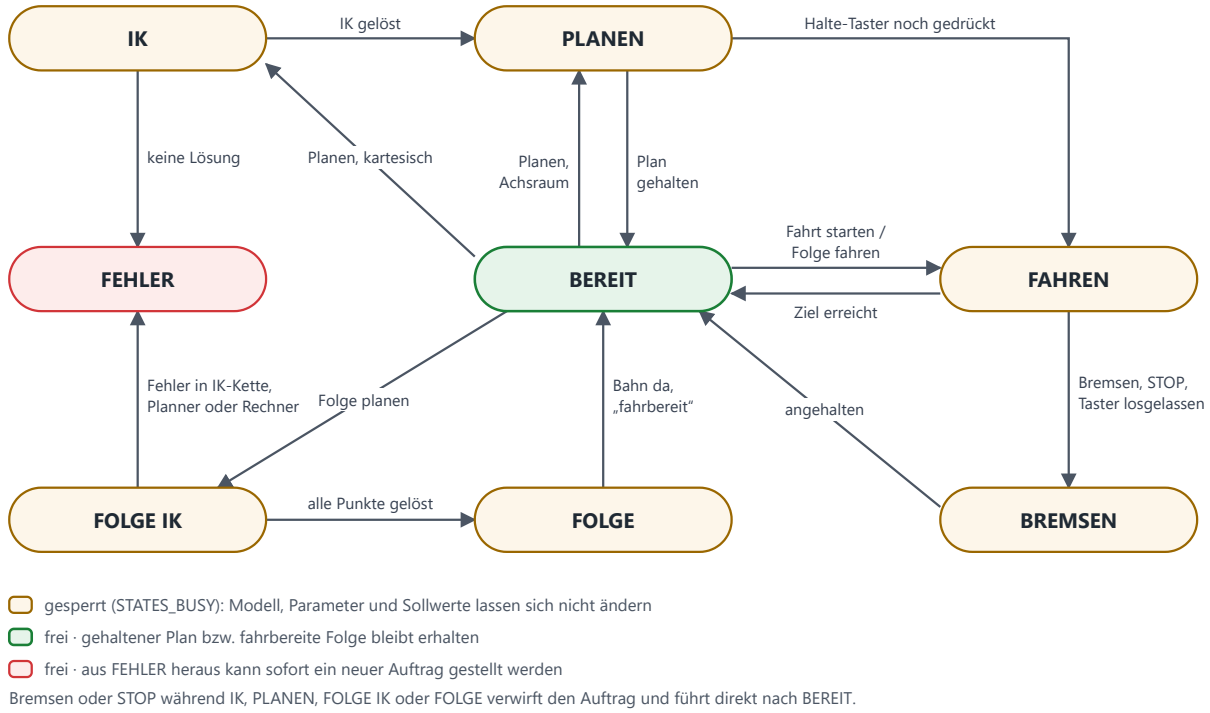
**Nichts wird still korrigiert.** Begrenzte Werte werden gelb gefärbt und gemeldet, verworfene Stützpunkte gezählt, Abweichungen stehen im Bericht und als `#`-Zeile in der CSV.

**Takte planen sich im `finally` neu.** `_tick` und `_record_tick` fangen jede Exception, melden sie und bestellen den nächsten Durchlauf im `finally`. Eine Exception kostet so einen Takt, nie den Takt. Dasselbe gilt im Executor-Thread über `RosBridge._guard()`.

**Eine Stelle je Zuständigkeit.** Zustand nur über `_set_state()`, Sperren nur über `_update_locks()`, Achswerte hinaus nur über `_push_positions()`, Modelländerungen nur über `_commit()`, Fahrten nur über `_start_playing()`.

**Radius 0 heißt Genauhalt** – in allen drei Verfahren.

## 3.4 State Machine




*Zustände der Ablaufsteuerung (`bahn.STATE_*`)*

Die Zustände stehen als Zeichenketten in `bahn.py` (`BEREIT`, `IK`, `PLANEN`, `FOLGE IK`, `FOLGE`, `FAHREN`, `BREMSEN`, `FEHLER`). `UrdfViewer._set_state()` ist der einzige Weg, den Zustand zu wechseln; es setzt `lock_reason` für alle Zustände in `STATES_BUSY` und ruft `_update_locks()`, die einzige Stelle, an der Bedienelemente aktiviert oder gesperrt werden. Daneben gibt es zwei weitere Sperrgründe: STOP (`self.stopped`, sperrt Sollwerte) und den Hochlauf von `move_group` (`lock_reason = "Neustart move_group"`).

`_brake(why)` ist die einzige Bremse. Je nach Zustand verwirft sie eine laufende IK, Planung, IK-Kette oder Kindprozess-Rechnung oder leitet beim Fahren die Bremsrampe ein (`TrajectoryPlayer.brake()`, Zeitrate in `BRAKE_TIME_S` = 0,4 s linear auf null). Sie stoppt zugleich das Joggen und lässt den Halte-Taster fallen.

## 3.5 Takte und Timer

| Takt | Ort | Periode | Aufgabe |
|---|---|---|---|
| `_tick` | GUI-Thread (`after`) | `FOOTER_TICK_MS` = 50 ms | Fußzeile, Ergebnisse von `RosBridge` abholen, Kindprozesse abfragen (`_totg_poll`, `_intrinsic_poll`), Watchdogs für `move_group` und TF |
| `_motion_tick` | GUI-Thread | `IPO_TICK_MS` = 20 ms | einen Interpolationsschritt fahren |
| `_record_tick` | GUI-Thread | `RECORD_TICK_MS` = 40 ms (25 Hz) | Bahnpunkt anbieten |
| `_jogj_step` | GUI-Thread | `JOG_TICK_MS` = 50 ms | Achs-Dauerjoggen |
| `_hold_watch` | GUI-Thread | `HOLD_WATCH_MS` = 100 ms | Watchdog am Halte-Taster |
| TF-Timer | Executor | 1/`TF_RATE_HZ` = 33 ms | `/tf` und `/joint_states` senden |
| Auftrags-Timer | Executor | 1/`JOB_RATE_HZ` = 50 ms | einen Auftrag stellen, Timeouts prüfen |

| Takt | Ort | Periode | Aufgabe |
|---|---|---|---|
| Probe | Executor | 1 s | eigenen `move_group` und seine Services im Graphen suchen |

Die Bahnzeit des `TrajectoryPlayer` entsteht aus der Summe der Takte (`ticks × dt`), nicht aus der Wall-Clock-Zeit. Dadurch ist dieselbe Bahn reproduzierbar; die Abweichung zur Wall-Clock-Zeit wird am Ende jeder Fahrt gemeldet (Meldung „Wanduhr“).

# 4 Module im Einzelnen

Die folgenden Abschnitte nennen die öffentliche Schnittstelle jedes Moduls, also das, was andere Module aufrufen. Private Helfer (`_…`) sind weggelassen.

## 4.1 `einstellungen`, `stil`

Reine Konstanten. `einstellungen.py` enthält die Pfade, die TF-Rate, Recorder-Schwellen und Slider-Spannen; `stil.py` Farben und die Maße der Baumansicht. Vollständige Liste in Anhang A.

Seit Stand 2 stehen in `einstellungen.py` keine absoluten Pfade mehr. `HIER` ist der Ordner des Moduls; `URDF_PATH` (`bmKinematic.urdf`), `BACKUP_DIR` (`URDF_Speicher/`) und `BAHN_DIR` (`TCP_Fahrten/`) hängen daran und lassen sich über `URDF_GUI_URDF`, `URDF_GUI_SICHERUNG` und `URDF_GUI_BAHNEN` umlenken (eine leere Variable zählt als nicht gesetzt, `~` wird aufgelöst). `BAHN_VIEWER` nennt die Bahnansicht, auf die die GUI nach einem Export verweist.

## 4.2 `matrix`, `konvention`, `anzeige`

`matrix.py` arbeitet mit 4×4-Matrizen als Liste von Zeilen, bewusst ohne NumPy (feste Summationsreihenfolge, reproduzierbar): `identity()`, `multiply(a, b)`, `from_rpy_xyz(rpy, xyz)`, `from_axis_angle(axis, angle)` (Rodrigues), `from_translation(xyz)`, `invert(m)` (orthonormaler Sonderfall), `to_xyz_rpy(m)`, `to_quaternion(m)` (Shepperd, Rückgabe `x, y, z, w`).

`konvention.py` hält die drei Drehkonventionen für die Winkel A, B, C. Alle sind intrinsisch: `R = R_a1(A) · R_a2(B) · R_a3(C)`.

| Konvention | Achsen A, B, C | Bereich B | entartet bei B = |
|---|---|---|---|
| `ZYX` (Standard, Kardan, = URDF-rpy) | Z, Y, X | −90° … 90° | ±90° |
| `ZYZ` (Euler) | Z, Y, Z | 0° … 180° | 0°, 180° |
| `ZXZ` (Euler) | Z, X, Z | 0° … 180° | 0°, 180° |

Schnittstelle: `pruefe(name)`, `from_abc_xyz(abc, xyz, konvention)`, `to_xyz_abc(m, konvention)`, `from_rpy_xyz` / `to_xyz_rpy` (Brücke zur URDF, immer ZYX mit `(roll, pitch, yaw) = (C, B, A)`), `singularitaetsabstand`, `ist_singulaer`, `kennung` / `lies_kennung` (Kopfzeile `drehkonvention=…`), `spaltennamen(praefix, konvention)` (z. B. `a_z_deg, b_y_deg, c_x_deg`).

`anzeige.ANZEIGE` ist das einzige Exemplar von `Anzeigekonvention`. Wer etwas anzeigt, ruft `zerlege(matrix)`; wer eine Eingabe weiterreicht, ruft `baue(abc, xyz)`. `setzen(name)` schaltet um und ruft alle mit `melden()` eingetragenen Callbacks; `sperren(grund)` / `freigeben()` verhindern das Umschalten (genutzt während einer Aufzeichnung). In die URDF wird immer ZYX geschrieben – die Einstellung betrifft nur Anzeige und Eingabefelder.

## 4.3 `urdf_modell`

Arbeitet auf einem `xml.etree.ElementTree`-Baum (`xml_root`), der in der GUI die einzige Wahrheit über das Modell ist.

| Funktion | Zweck |
|---|---|
| `parse(path)` → `(root, name, links, joints, xml)` | Datei lesen; `links` / `joints` sind Listen von `dict` (siehe unten) |
| `parse_root(root)` → `(name, links, joints)` | dasselbe aus einem Baum |
| `serialize(root)` | eingerückter XML-Text |
| `validate_root(root)` → `[Fehlertext]` | Namen, Typen, Eltern, genau ein Root-Link, Kreise, Grenzen, Achsen, Kopplungen |
| `append_limb(root, link, joint, jtype, parent, xyz, rpy, axis, lower, upper)` | Glied an einen Link anhängen |
| `insert_limb(root, link, joint, jtype, below, xyz, rpy, axis, lower, upper, keep_pose)` | Glied vor dem Gelenk `below` einfügen |
| `insert_above_root(root, link, joint, xyz, rpy)` | neuen Basisframe über den Root-Link setzen |
| `remove_limb(root, joint, keep_pose)` | Glied herausnehmen, Nachfolger rücken auf |
| `remove_branch(root, link_names)` | Teilbaum löschen |
| `rename_link`, `rename_joint` | umbenennen mit allen Verweisen (auch `<mimic>`) |
| `set_axis`, `set_limit`, `set_mimic`, `clear_mimic`, `set_visual` | Einzelattribute setzen |
| `coupling_problem(joints, name, source)` | Kopplung prüfen (keine Ketten, keine festen Gelenke) |
| `apply_couplings(joints, positions)` | Werte gekoppelter Gelenke eintragen (`faktor · führung + versatz`, begrenzt) |
| `slider_range`, `slider_step`, `snap`, `value_digits`, `limit_digits` | Slider und Anzeigegenauigkeit |
| `save_urdf(path, xml)` → Sicherungspfad | atomar schreiben; vorhandene Datei vorher mit Zeitstempel nach `BACKUP_DIR` kopieren |
| `backup_dir_for(path)`, `bahn_dir_for(path)`, `timestamped_name(path)` | Ablageorte |

Ein Gelenk-`dict` enthält `name, type, parent, child, xyz, rpy, origin (4×4), quat, axis (normiert), lower, upper, effort, velocity, has_limit, mimic` (`None` oder `{"joint", "multiplier", "offset"}`). Ein Link-`dict` enthält `name, mass, com, has_inertial, visual`. `<origin>` wird mit 10 Stellen geschrieben (`ORIGIN_DIGITS`), damit Umrechnungen beim Einfügen und Herausnehmen nicht als Lagefehler akkumulieren.

## 4.4 `kinematik`

`forward_kinematics(links, joints, positions)` → `{link: 4×4}` relativ zur ersten Root-Link; nicht erreichbare Teilbäume fehlen im Ergebnis. `local_transform(joint, value)` gibt Eltern→Kind bei Gelenkwert zurück (auch von `RosBridge` für `/tf` benutzt). `root_links`, `leaf_links` (TCP-Kandidaten) und `tree_layout` (Zwei-Pass-Layout für die Baumansicht) ergänzen das Modul.

## 4.5 `moveit_konfig`

**`MoveItConfig`** hält alle Werte für `move_group`, getrennt nach Wirkung:

| Attribut | Inhalt | wirkt |
|---|---|---|
| `plan`, `planner_id` | Felder aus `PLAN_FIELDS` (Overrides in %, Planungszeit, Versuche, Achs-Zieltoleranz in Grad, Überschleifradius) und Fahrart | je Auftrag, sofort |

| Attribut | Inhalt | wirkt |
|---|---|---|
| `limits` | `{gelenk: {velocity, acceleration, deceleration, jerk}}`, `None` = Grenze aus | nach Neustart |
| `cartesian` | `max_trans_vel`, `max_trans_acc`, `max_trans_dec`, `max_rot_vel` | nach Neustart |
| `planner` | `sampling_time` (nur mit gepatchtem Pilz) | nach Neustart |
| `solver`, `kinematics`, `kdl`, `kdl_position_only` | IK-Solver und Parameter | nach Neustart |
| `group`, `tip` | Planning Group (`manipulator`), Tip-Link (leer = einziger Leaf-Link) | nach Neustart |
| `use_ruckig`, `check_collisions`, `allow_execution` | Adapter der Pipeline | nach Neustart |

Wichtige Methoden: `sync_joints(joints)` (Grenzen an das Modell anpassen, fehlende aus der URDF ableiten: a = 2·v, Jerk = 10·a), `params_yaml(urdf, srdf)` (vollständige Parameterdatei, Abschnitt 5.3), `startup_signature()` (Text aller neustartpflichtigen Werte; ändert er sich, ist `move_group` „veraltet“).

**`build_srdf(name, links, joints, group, tip)`** erzeugt die SRDF neu aus dem Modell: eine Planning Group als Kette Root-Link→Tip, ein `group_state` „null“, und `disable_collisions` für jedes benachbarte Linkpaar.

**`MoveItProcess`** startet und beendet `move_group`. `start(config, urdf, srdf)` schreibt `move_group.yaml`, `robot.urdf`, `robot.srdf` in einen neuen Ordner `/tmp/urdf_gui_moveit_*`, sucht das Binary über `AMENT_PREFIX_PATH` (Rückfall: `ros2 run`) und startet mit `--params-file` und eindeutigem Node-Namen. Ausgabe nach `move_group.log`. `stop()` signalisiert die ganze Prozessgruppe (SIGINT 3 s → SIGTERM 3 s → SIGKILL 2 s). `state()` liefert `aus | startet | laeuft | beendet | fehler`; `strays()` / `kill_strays()` finden bzw. beenden fremde `move_group` über `/proc`; `sweep_workdirs()` räumt alte Arbeitsordner (beim GUI-Start alles älter als 1 h).

## 4.6 bahn

- **Zustände** `STATE_*`, `STATES_BUSY`; Zielräume `SPACE_JOINT`, `SPACE_CART`.
- **`moveit_error_text(code)`** übersetzt `MoveItErrorCodes` in einen Satz mit Hinweis (Tabelle in Teil II, Abschnitt 11.1).
- **`hermite(p0, v0, p1, v1, span, s)`** – kubische Hermite-Interpolation; nutzt die von Pilz/TOTG gelieferten Geschwindigkeiten, damit die Geschwindigkeit an den Stützpunkten stetig bleibt.
- **`TrajectoryPlayer(points, dt=IPO_DT)`** mit `points = [(t, {name: pos}, {name: vel}), …]`: `step()` → Sollwerte nach einem Takt, `sample(t)`, `brake()`, Eigenschaften `done`, `stopped`, `progress`, `wall_drift(elapsed)`.
- **`bahn_startabweichung(punkte, stand)`** → `(achse, betrag, fehlende_achsen)` – vergleicht den ersten Stützpunkt mit dem Achsstand (Prüfung vor jeder Folgenfahrt mit `FOLGE_START_TOL` = 1e-4 rad).
- **`PathRecorder`** – Ringpuffer (max. 20 000 Punkte) für die TCP-Bahn. `start(joint_names, frame_names, chain, couplings, robot_name, joint_tolerances, drehkonvention)`, `sample(xyz, abc, values, frames, now)` (filtert Stillstand nach Weg, Drehung und Achsänderung), `stop()`, `clear()`, `reset()`, `header()`, `write_csv(path, zusatz=())`.

## 4.7 `ros_bruecke.RosBridge`

| Methode | Thread | Zweck |
|---|---|---|
| `start()` → `None` oder Meldung | GUI | rclpy initialisieren, Node `urdf_gui_tf`, Publisher, Timer, Executor-Thread starten; importiert ROS erst hier |
| `shutdown()` | GUI | Thread beenden, Node zerstören |
| `set_model(joints, urdf_xml)` | GUI | `robot_description` senden, Liste der TF-Gelenke setzen, einmal sofort senden |
| `set_positions(positions)` | GUI | Sollwerte übergeben (Lock) |
| `request_plan(payload)`, `request_ik(payload)`, `request_sequence(payload)` | GUI | Auftrag ablegen (siehe 5.2) |
| `reset_moveit(node_name)` | GUI | nach Start/Stopp von `move_group`: Clients verwerfen, erwarteten Node-Namen setzen |
| `take_results()` → `[(art, wert)]` | GUI | Ergebnisse in einem Zug abholen |
| `running`, `thread_alive()`, `moveit_ready`, `sequence_ready`, `last_publish`, `cycle_ms`, `enabled` | GUI (lesend) | Zustand für Anzeige und Sperren |

Pro Zeitpunkt ist höchstens **ein** Service-Aufruf offen (`_pending`); er endet spätestens nach dem mitgegebenen Timeout (`timeout`) mit einer Fehlermeldung.

## 4.8 `totg_prozess.TotgProzess`

Generischer Starter für einen Rechner-Kindprozess; die GUI hält zwei Exemplare (`self.totg` mit `totg_rechner.py`, `self.intrinsic` mit `intrinsic_lin_rechner.py`).

| Methode | Zweck |
|---|---|
| `TotgProzess(rechner=TOTG_RECHNER)` | Pfad zum Rechnerskript |
| `starten(auftrag, tag, frist)` → `None` oder Meldung | Ordner `/tmp/urdf_gui_totg_*` anlegen, `auftrag.json` schreiben, Prozess starten (eigene Session, Ausgabe nach `totg.log`) |
| `abfragen()` | `None` solange er läuft; sonst `{"tag", "text", "dauer"}` oder `{"tag", "error"}` (auch bei Timeout) |
| `protokoll(lines)` | letzte Zeilen von `totg.log` |
| `abbrechen()` | Prozessgruppe beenden (SIGTERM, dann SIGKILL), Ordner bleibt |
| `aufraeumen()` | beenden und Ordner löschen |
| `laeuft` | läuft ein Prozess? |

## 4.9 `sequenz`

Liest und prüft Punktlisten; keine Oberfläche, kein ROS.

- `lies_liste(text)` → `(punkte, befunde)`; `lies_zeile(text)` → `Punkt` (Format in Abschnitt 5.8).
- `Punkt(xyz, abc, achsen, name, fahrart, tempo, beschleunigung, radius, hilfspunkt, achsziel)`; `ergaenze_lage(xyz, abc)` nur bei Achszielen.
- `Folge(punkte, modal)`: `aufloesen()` → Tupel von `Schritt` mit `herkunft` je Feld (`zeile`, `modal`, `achsziel`); `pruefe(schritte, kugeln_pruefen)` → Tupel von `Befund`; `fehler()`, `tragfaehig()`; `nutzlast(gruppe, start, toleranz, planungszeit, versuche, tag, kugeln_pruefen)` → Auftrag für `/plan_sequence_path`

(Abschnitt 5.2); jeder Punkt muss dafür Achswerte tragen, eine Lage ohne Achswerte ist ein `ValueError`; `bericht()`, `spielraum()`, `endzustand()`.

- `Befund(stufe, text, stelle, paar)` mit den Stufen `fehler` (hält an), `warnung`, `hinweis`.
- `loese_achsziel(punkt, gelenke, index)` ordnet die Werte einer J-Zeile den Gelenken zu und prüft Anzahl und Grenzen (ohne Begrenzung).
- `bereinige_zeitreihe(punkte)` entfernt Stützpunkte ohne wachsende Zeit (bekanntes Verhalten beim Überschleifen in MoveIt), `pruefe_bahn(punkte)` liefert Befunde zur Antwort.

## 4.10 `totg_folge`

- `pruefe_parameter(path_tolerance, resample_dt)`.
- `anfrage(nutzlast, gelenke, drehend, params_datei, path_tolerance, resample_dt, min_angle_change)` → `(auftrag, befunde)`; baut aus der Nutzlast eine Polylinie im Joint Space, zerlegt sie an Genauhalten (Radius 0) und Umkehrpunkten (`abschnitte`, `ist_umkehr` – gleiche Regel wie `Path::create` in MoveIt), meldet verworfene Punkte (`doppelte_punkte`) und lehnt wechselnde Overrides ab.
- `auftrag_text(auftrag)`, `antwort_lesen(text, gelenke)` → Ergebnis in derselben Form wie `RosBridge._read_sequence` plus Block `totg`.
- `endabweichung(punkte, auftrag)`, `eckwerte(zeiten, orte, ecken)` (kleinster Abstand der TCP-Bahn zu jeder Ecke und Geschwindigkeit dort), `zusatzzeilen(herkunft)` (Kommentarzeilen für die CSV).

## 4.11 `intrinsic_folge`

- `wrapper_pfad()` – `$INTRINSIC_WRAPPER` oder `WRAPPER_STANDARD`.
- `pruefe_parameter(rundung_rot, resample_dt, ueberschleifen_rad, gelenk_abtastung_rad, tcp_v_max, tcp_a_max)`; `bahngrenze_lesen(roh)` (leer → `None`).
- `anfrage(nutzlast, gelenke, params_datei, tip, rundung_rot, resample_dt, wrapper, grenzen_lesen, ueberschleifen_rad, gelenk_abtastung_rad, tcp_v_max, tcp_a_max)` → `(auftrag, befunde)`. Regeln: nur LIN und PTP; ein Override für die ganze Bahn; zwischen zwei LIN-Zeilen ein einziger Radius > 0 für die ganze Kurve; Abschnittsgrenzen an Genauhalt, Umkehr und Wechsel der Fahrart; ein Wechsel mit Radius > 0 wird im Joint Space überschliffen (`verschleifen`). Joint Limits kommen über `intrinsic_lin_auftrag.grenzen_lesen` aus der Parameterdatei von `move_group` (Beschleunigungsgrenze ist Pflicht).
- `antwort_lesen(text, gelenke)` – Grundgerüst über `totg_folge.antwort_lesen`, dazu Block `intrinsic` mit Kennwerten.
- `zusatzzeilen(herkunft)` – TOTG-Zeilen plus Intrinsic-Schlüssel (`ZUSATZ_SCHLUESSEL`).

## 4.12 Rechner-Kindprozesse

**`totg_rechner.py`** liest den Auftrag, schreibt die Parameterdatei für MoveItPy um (`params_fuer_moveitpy`: `planning_pipelines` als `pipeline_names`, eigener Name für den Planning Scene Monitor), startet `MoveItPy`, ruft je Abschnitt `apply_totg_time_parameterization` und hängt die Abschnitte an (`verketten`).

**`intrinsic_lin_rechner.py`** bildet den LIN-Planner von Intrinsic Core nach, mit IK aus MoveIt. Je LIN-Abschnitt:

1. FK der Wegpunkte → Posen des Tip-Links (MoveIt),
2. Wrapper `lin_kurve`: verrundete kartesische Kurve, gleichmäßig abgetastet,
3. IK-Kette an allen Kurvenposen, Startwert jeweils die vorige Lösung,
4. Wrapper `lin_neuabtastung` (Resampling): Kurvenparameter, gleichabständig im Joint Space,
5. IK-Kette an den dichten Posen; behalten werden die vom Wrapper genannten Indizes.

PTP-Abschnitte gehen ohne IK als Gelenkpfad weiter. Danach (optional `toppra_proben` + TCP-Ableitungen per MoveIt-FK für `tcp_v_max`/`tcp_a_max`) rechnet der Wrapper im Modus `toppra` die Time Parameterization über alle Teilpfade. Prüfungen nach Intrinsic (`MAX_IK_SPRUNG_RAD` = π/4, `START_TOLERANZ_RAD`, `ZIEL_TOLERANZ_RAD`) und eine

eigene (FK der IK-Lösung muss die Pose auf 1 mm / 0,01 rad treffen) führen zum Abbruch, nie zur Korrektur. Mit `ast_folgen` (Standard an) wird eine LIN-Kette, die die Zielpose auf einem anderen IK-Ast erreicht, fortgesetzt und im Bericht gemeldet.

Beide Rechner schreiben die Antwort atomar (Zwischendatei + `os.replace`) und beenden sich mit `os._exit`, weil MoveItPy beim normalen Ende hängen kann.

**`intrinsic_lin_auftrag.py`** liefert `parameter_lesen(pfad)`, `grenzen_lesen(params, gelenke)` (Positionsgrenzen aus der URDF, v/a aus `robot_description_planning.joint_limits`; `continuous` → ±1e9) und `tip_aus_srdf`. Als Skript macht es aus einem TOTG-Auftrag einen LIN-Auftrag (`--wrapper`, `--tip`, `--rundung-t`, `--rundung-r`).

## 4.13 `urdf_gui.UrdfViewer`

Die Klasse erbt von `tk.Tk` und ist nach Themen gegliedert (Abschnitte durch Kommentarbalken getrennt):

| Bereich | Einstieg | Inhalt |
|---|---|---|
| Aufbau Hauptfenster | `_build_ui`, `_build_head`, `_build_body`, `_build_foot` | Karten, Tabellen, Reiter |
| Meldungen, STOP | `_log`, `_toggle_stop` | Meldungsfenster (max. 600 Zeilen), STOP/FREI |
| Fahrfenster | `_open_drive`, `_build_drive_*`, `_drive_param_commit` | Zielfelder, Fahrparameter, Auftragstasten |
| Sperren, Zustand | `_lock`, `_update_locks`, `_set_state`, `_job_*` | siehe 3.3/3.4 |
| Einzelauftrag | `_drive_plan`, `_plan_joint_goal`, `_on_plan_result`, `_ik_request`, `_on_ik_result` | Planen über Pilz, IK über `/compute_ik` |
| Joggen | `_jog_press/_jog_step/_on_jog_result`, `_jogj_*` | kartesisch über IK, Achse direkt |
| Fahren | `_drive_start`, `_start_playing`, `_motion_tick`, `_finish_playing`, `_brake` | Abtasten im 20-ms-Takt |
| Halte-Taster | `_hold_press`, `_hold_release`, `_hold_watch`, `_hold_launch` | planen und fahren, solange gedrückt |
| `move_group` | `_moveit_start`, `_moveit_stop`, `_moveit_watch`, `_moveit_stale` | Kindprozess und Watchdog |
| Parameterfenster | `_open_params`, `_build_param_*`, `_params_apply`, `_params_restart` | sechs Reiter |
| Laden/Speichern | `_load`, `_save`, `_save_as`, `_discard` | Datei und Sicherung |
| Verlauf | `_commit`, `_undo`, `_push_history` | einzige Änderungsstelle, 25 Schritte zurück |
| Sollwerte | `_apply`, `_on_slider`, `_push_positions` | direkt stellen ohne MoveIt |
| Recorder | `_toggle_record`, `_record_tick`, `_export_record` | Aufzeichnung und CSV |
| Punktfolge | `_open_sequence`, `_seq_read`, `_seq_plan`, `_seq_ik_next`, `_seq_send`, `_totg_send`, `_intrinsic_send`, `_on_sequence_result`, `_seq_drive`, `_seq_export` | siehe 6.6 |
| Kinematik-Anzeige | `_tcp_link`, `_tcp_pose`, `_pose_text`, `_update_kinematics` | TCP-Zeile, Frametabelle |
| Editor | `_open_editor`, `_build_tree_panel`, `_draw_tree`, `_edit_apply`, `_open_add/insert/remove/frame_window`, `_build_visual_tab` | Bearbeiten-Fenster |
| Takt | `_tick`, `_tick_rumpf`, `_on_close` | Fußzeile, Ergebnisse, Aufräumen |

Der aktive TCP (`_tcp_link`) ist `moveit_config.tip`, falls gesetzt und vorhanden, sonst der einzige Leaf-Link außer dem Root-Link; bei mehreren Leaf-Links ohne Wahl gibt es keinen TCP (Anzeige und Recorder melden das).

### 4.14 `bahn_viewer3`

Eigenständige Tk-Anwendung `BahnViewer` ohne ROS und ohne URDF. `Track.load(path)` liest eine CSV (alte Spalten `roll/pitch/yaw` und neue `a_*/b_*/c_*`), bestimmt die Drehkonvention aus `#drehkonvention` und den Spaltennamen und bricht bei Widerspruch ab. Gezeichnet wird mit eigener orthographischer Projektion (`Camera`) auf einer Tk-Canvas, ohne OpenGL. Mehrere Bahnen lassen sich gleichzeitig laden; die aktive gibt den Takt vor, die Marker der anderen laufen zur gleichen Zeit seit Bahnbeginn mit. `--pruefen datei.csv …` liest nur ein und berichtet.

## 5 Schnittstellen

### 5.1 ROS 2

| Richtung | Name | Typ | Einzelheiten |
|---|---|---|---|
| Publisher | `/tf` | `tf2_msgs/TFMessage` (`TransformBroadcaster`) | alle Gelenke mit Eltern und Kind, **auch feste** (bewusst nicht `/tf_static`), 30 Hz, gemeinsamer Zeitstempel |
| Publisher | `robot_description` | `std_msgs/String` | QoS: depth 1, reliable, transient local; bei jedem Modellwechsel |
| Publisher | `/joint_states` | `sensor_msgs/JointState` | alle beweglichen Gelenke inkl. gekoppelter, nur `position`, im TF-Takt |
| Client | `/plan_kinematic_path` | `moveit_msgs/GetMotionPlan` | Einzelauftrag, Pipeline `pilz_industrial_motion_planner`, Achsziele als `JointConstraint` |
| Client | `/compute_ik` | `moveit_msgs/GetPositionIK` | Lage im Frame des Root-Links, Startwert = Achsstand bzw. Vorgänger, `avoid_collisions` = Kollisionsprüfung |
| Client | `/plan_sequence_path` | `moveit_msgs/GetMotionSequence` | Punktfolge (Pilz), Capability `MoveGroupSequenceService` |

Node-Namen: GUI `urdf_gui_tf`, `move_group` als `move_group_gui<PID>_<laufende Nummer>`, Rechner `urdf_gui_totg_<PID>` bzw. `urdf_gui_intrinsic_lin_<PID>`. Die Probe hält `moveit_ready` für wahr, wenn die eigene Node im Graphen steht und `/plan_kinematic_path` sowie `/compute_ik` angeboten werden; `sequence_ready` zusätzlich bei `/plan_sequence_path`.

### 5.2 GUI ↔ `RosBridge`

Alle Nutzlasten sind `dict` aus Zahlen, Zeichenketten, Tupeln und `dict`. Winkel in rad, Längen in m, Overrides als Faktor 0…1.

**`request_plan`**

```
{"group": "manipulator", "planner_id": "PTP" | "LIN" | "CIRC",
 "start": {gelenk: wert}, "goal": {gelenk: wert},
 "tolerance": rad, "velocity_scaling": 0..1, "acceleration_scaling": 0..1,
 "planning_time": s, "attempts": n,
 "timeout": planning_time * attempts + PLAN_TIMEOUT_MARGIN_S, "tag": n}
```

**`request_ik`**

```
{"group": ..., "tip": link, "frame": wurzel_link,
 "position": (x, y, z), "orientation": (qx, qy, qz, qw),
 "seed": {gelenk: wert}, "avoid_collisions": bool,
 "solver_timeout": s, "timeout": solver_timeout * attempts + 5.0, "tag": n}
```

**`request_sequence`** (gebaut von `sequenz.Folge.nutzlast`)

```
{"group": ..., "timeout": s, "tag": n,
 "items": [{"group", "planner_id", "tolerance", "velocity_scaling",
            "acceleration_scaling", "planning_time", "attempts",
            "blend_radius", "name",
            "goal": {gelenk: wert},          # Achsziel, Pflicht
            "start": {gelenk: wert}},        # nur im ersten Posten
           ...]}
```

Ein Posten ohne `goal` wird von `RosBridge` nicht gestellt, sondern sofort als `{"tag", "error": "Posten ohne Achsziel: …"}` zurückgemeldet.

**Antworten** aus `take_results()`:

| Art | Wert |
|---|---|
| `"plan"` | `{"tag", "code", "points": [(t, {pos}, {vel})], "planning_time"}` oder `{"tag", "error"}` |
| `"ik"` | `{"tag", "code", "values": {gelenk: wert}}` oder `{"tag", "error"}` |
| `"sequence"` | `{"tag", "code", "points", "teilbahnen": [n je Teilbahn], "planning_time"}` oder `{"tag", "error"}` |
| `"moveit"` | `bool` – eigener `move_group` bereit |
| `"sequence_ready"` | `bool` – `/plan_sequence_path` vorhanden |
| `"moveit_nodes"` | Liste, wenn mehr als ein `move_group` im Graphen steht |
| `"ros_error"` | Text einer abgefangenen Exception im Executor |

`code` ist ein `MoveItErrorCodes`-Wert (1 = Erfolg). Teilbahnen einer Pilz-Folge werden in `_read_sequence` zu einer Zeitreihe verkettet; der doppelte Übergangspunkt fällt dabei weg.

## 5.3 `move_group`: Parameterdatei und SRDF

`MoveItConfig.params_yaml()` erzeugt (gekürzt):

```
/**:
  ros__parameters:
    robot_description: |-
      <robot …>              # URDF unverändert
    robot_description_semantic: |-
      <robot …>              # SRDF aus build_srdf
    robot_description_kinematics:
      manipulator:
        kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
        kinematics_solver_timeout: 0.05
        kinematics_solver_search_resolution: 0.005
        max_solver_iterations: 500          # nur bei KDL, Ganzzahl!
        epsilon: 1.0e-05
        orientation_vs_position: 1.0
        position_only_ik: false
    robot_description_planning:
      joint_limits:
        joint1: {has_velocity_limits: true, max_velocity: 1.0, …,
                 has_deceleration_limits: true, max_deceleration: -2.0}
      cartesian_limits: {max_trans_vel: 1.0, max_trans_acc: 2.25,
                         max_trans_dec: -5.0, max_rot_vel: 1.57}
    planning_pipelines: ["pilz_industrial_motion_planner"]
    default_planning_pipeline: pilz_industrial_motion_planner
    pilz_industrial_motion_planner:
      planning_plugins: [pilz_industrial_motion_planner/CommandPlanner]
      request_adapters: [ResolveConstraintFrames, ValidateWorkspaceBounds,
                         CheckStartStateBounds, CheckStartStateCollision*]
      response_adapters: [AddRuckigTrajectorySmoothing*, ValidateSolution*,
                          DisplayMotionPath]
      sampling_time: 0.1
    capabilities: "…MoveGroupSequenceAction …MoveGroupSequenceService"
    allow_trajectory_execution: false
    moveit_manage_controllers: false
    publish_planning_scene: true  # usw.
```

`*` = abhängig von „Ruckig" bzw. „Kollisionsprüfung". Zahlen werden über `yaml_scalar` immer so geschrieben, dass rclcpp sie als `double` liest (`2.0` statt `2`, `1.0e-05` statt `1e-05`); eine Ganzzahl an einer double-Stelle beendet `move_group`. Dieselbe Datei lesen TOTG- und Intrinsic-Rechner (über `params_fuer_moveitpy` umgeschrieben).

Die SRDF hat genau eine Planning Group (`manipulator`) als Kette Root-Link → Tip, einen Group State „null" und `disable_collisions` für benachbarte Links.

## 5.4 Kindprozess-Protokoll

```
python3 <rechner>.py AUFTRAG.json ANTWORT.json   # rechnen
python3 <rechner>.py --pruefen [WRAPPER]         # nur Umgebung prüfen
```

- Arbeitsordner `/tmp/urdf_gui_totg_*` (Präfix für beide Rechner), darin `auftrag.json`, `antwort.json`, `totg.log` (stdout+stderr).
- Die Antwort ist JSON mit `"ok": true|false`; bei Fehler `"fehler": "<Typ>: <Text>"`. Sie wird atomar geschrieben, eine halbe Datei kann die GUI nicht lesen.
- Timeout: `totg_folge.FRIST_S` = 90 s bzw. `intrinsic_folge.FRIST_S` = 300 s; danach beendet `TotgProzess` die Prozessgruppe.
- Der Ordner des letzten Auftrags bleibt bis zum nächsten Auftrag liegen (Fehlersuche), beim Schließen der GUI wird er gelöscht.

## 5.5 TOTG: Auftrag und Antwort

```
// auftrag.json (totg_folge.anfrage)
{"stand": 2, "params_datei": "/tmp/urdf_gui_moveit_x/move_group.yaml",
 "gruppe": "manipulator", "gelenke": ["joint1", …],
 "start": {"joint1": 0.0, …},           // inkl. gekoppelter Gelenke
 "namen": ["Start", "P1", …],
 "wegpunkte": [[q1, q2, …], …],          // Start + je Posten ein Achsziel
 "abschnitte": [[0, 3], [3, 7]],         // Indizes in wegpunkte
 "umkehr": [3],
 "velocity_scaling": 1.0, "acceleration_scaling": 1.0,
 "path_tolerance": 0.1, "resample_dt": 0.01, "min_angle_change": 0.001}

// antwort.json (totg_rechner.rechne)
{"ok": true, "gelenke": [...], "punkte": [[t, [pos…], [vel…]], …],
 "dauer_s": 4.08, "teilbahnen": [n, …], "eingang": 8,
 "rechenzeit_s": 0.02, "start_s": 3.1,
 "moveit_version": "2.12.4", "moveit_pfad": "…"}
```

## 5.6 Intrinsic: Auftrag und Antwort

Der Auftrag (Stand 6) enthält zusätzlich zu den TOTG-Feldern `wrapper`, `tip`, `fahrarten` (je Abschnitt `LIN`/`PTP`), `verschleifen` (Wegpunkt-Indizes ohne Halt), `ast_folgen`, `ueberschleifen_rad`, `gelenk_abtastung_rad`, `rundung_translation_m`, `rundung_rotation_rad`, `grenzen` (`min_position`, `max_position`, `max_velocity`, `max_acceleration` je Gelenk), optional `tcp_v_max`, `tcp_a_max` und – von der GUI ergänzt – `ik_timeout_s`. Weitere optionale Felder liest der Rechner mit Standardwert: `ik_toleranz_m`, `ik_toleranz_rad`, `abtastung_translation_m`, `abtastung_rotation_rad`, `roh_ausgeben`.

Die Antwort ist die Antwort des Wrappers im Modus `toppra` (Grundgerüst wie bei TOTG: `ok`, `gelenke`, `punkte`, `teilbahnen`, `rechenzeit_s` …), ergänzt vom Rechner um `rechner`, `lin_stand`, `fahrarten`, `verschliffen`, `ast_folgen`, `ast_gewechselt`, `lin` (Kennwerte je Abschnitt: Posenzahlen, größter IK-Sprung, Posenfehler, Ziel-/Startabweichung, Hinweise), `lin_vorgaben`, `herkunft`, `ik_aufrufe`, `ik_rechenzeit_s`, `start_s`, `moveit_version`, `moveit_pfad`, `hinweise` und – mit Bahngrenzen – `tcp_pruefung` (gemessene Höchstwerte von Tip-Geschwindigkeit und -Beschleunigung, Faktor zur Vorgabe, eingehalten ja/nein). `intrinsic_folge.antwort_lesen` bündelt das im Block `intrinsic`.

## 5.7 Intrinsic-Wrapper

```
intrinsic_toppra_wrapper AUFTRAG.json ANTWORT.json
intrinsic_toppra_wrapper --pruefen          # gibt u. a. "Stand N," aus
```

| Modus (`"modus"`) | Eingabe | Ausgabe |
|---|---|---|
| `lin_kurve` | `posen` ([x,y,z,qx,qy,qz,qw]), `rundung_translation_m`, `rundung_rotation_rad`, `abtastung_translation_m`, `abtastung_rotation_rad` | `posen` (abgetastete Kurve), `kurvenparameter`, `kurve`, `hinweise` |
| `lin_neuabtastung` | wie oben + `kurvenparameter`, `gelenkpfad`, `gelenk_abtastung_rad` | `posen` (dicht), `behalten_indizes`, `gelenk_abtastschritt_rad` |
| `toppra_proben` | wie `toppra` | `teilpfade` mit `q`, `qp`, `qpp`, `pruefsumme` je Probe |
| `toppra` | `gelenke`, `teilpfade`, `namen`, `ueberschleifen_rad`, Overrides, `resample_dt`, `grenzen`, optional `tcp_x`/`tcp_xp`/`tcp_xpp`/`tcp_pruefsummen`, `tcp_v_max`, `tcp_a_max`, `roh_ausgeben` | `punkte`, `beschleunigungen`, `abschnitte`, `grenzpruefung`, optional `roh`, `hinweise` |

Jede Antwort trägt `"ok"`; bei `false` steht der Grund in `"fehler"`. Die Dateien liegen in einem Unterordner `lin_*` des Rechner-Arbeitsordners (`a<n>_lin_kurve_auftrag.json` usw.). Mindeststand des Wrappers: 3.

## 5.8 Dateiformat Punktliste

```
zeile      := kommentar | lage | achsziel
kommentar  := "#" beliebig            (auch am Zeilenende; Leerzeilen erlaubt)
lage       := X Y Z A B C { feld }     X,Y,Z in m; A,B,C in Grad, Anzeigekonvention
achsziel   := ("J"|"j") wert { wert } { feld_j }
               ein Wert je beweglicher, nicht gekoppelter Achse (Grad bzw. m)
feld       := "fahrart=" (PTP|LIN|CIRC) | "tempo=" % | "beschleunigung=" %
            | "radius=" m | "name=" text
feld_j     := "tempo=" % | "beschleunigung=" % | "name=" text
```

Felder sind modal: ein gesetzter Wert gilt für alle folgenden Zeilen, bis er neu gesetzt wird. Den Anfangszustand liefert in der GUI das Parameterfenster (Fahrart, beide Overrides, Überschleifradius), nicht `sequenz.MODAL_START`. Achsziele fahren immer PTP mit Radius 0 und lassen den Modalzustand von Fahrart und Radius unberührt.

## 5.9 Dateiformat Bahn-CSV

Trennzeichen `;`, Dezimalpunkt. Vor dem Spaltenkopf stehen Kommentarzeilen `#schlüssel;wert…`, die der Viewer liest oder überliest:

```
#roboter;my_robot
#erzeugt;2026-10-05 13:23:09
#rate_hz;25
#drehkonvention;ZYX
#tcp;tcp11
#verfahren;Intrinsic            ← nur geplante Bahnen (zusatzzeilen)
#override_v;100.0               …  #dauer_s, #resample_dt, #teilbahnen, #verworfen,
#punkt;N00007;PTP;0.1               #endabweichung, #moveit_version, #punkt, …
#ecke_spalten;name;abstand_mm;v_mm_s;zeit_s
#ecke;N00008;33.0036;570.239;1.7532
#kette;link1;base_link;joint1   ← je Gelenk: Kind, Eltern, Gelenk
#kopplung;joint7;joint6;-1;0    ← Gelenk, Führung, Faktor, Versatz
t_s;x_m;y_m;z_m;a_z_deg;b_y_deg;c_x_deg;joint1_deg;…;base_link_x_m;…;tcp11_c_x_deg
```

Spalten: Zeit, TCP-Lage (m) und -Winkel (Grad, Konvention aus der Kennung, Achse im Namen), je beweglichem Gelenk ein Wert (`_deg` bzw. `_m`), danach je Frame sechs Spalten; der TCP-Frame steht zuletzt. Aufzeichnungen heißen `bahn_<JJJJMMTT_hhmmss>.csv`, geplante Bahnen `plan_<verfahren>_<…>.csv`.

## 5.10 Dateien und Ablageorte

| Ort | Inhalt | angelegt von |
|---|---|---|
| `BACKUP_DIR` (`URDF_Speicher/`, `$URDF_GUI_SICHERUNG`) | Sicherung der überschriebenen URDF mit Zeitstempel | `save_urdf` |
| `BAHN_DIR` (`TCP_Fahrten/`, `$URDF_GUI_BAHNEN`) | Vorschlagsordner für CSV-Export | `bahn_dir_for` |
| `archiv/` | frühere Fassungen, alte Tests, frühere Arbeitsdateien des LIN-Rechners | – |
| `/tmp/urdf_gui_moveit_*` | `move_group.yaml`, `robot.urdf`, `robot.srdf`, `move_group.log` | `MoveItProcess.start` |
| `/tmp/urdf_gui_totg_*` | `auftrag.json`, `antwort.json`, `totg.log`, `*_params.yaml`, `lin_*/` | `TotgProzess.starten` und Rechner |

# 6 Abläufe

## 6.1 Programmstart

1. Import: `totg_folge.STAND ≥ 2`, `intrinsic_folge.STAND == 1`, danach `pruefe_stand()` für die zehn Basismodule – bei Abweichung Abbruch mit Liste.
2. `UrdfViewer.__init__`: Zustand anlegen, Schriften und Stil setzen, Fenster bauen, Callback für die Drehkonvention eintragen.
3. Alte Arbeitsordner (> 1 h) entfernen, fremde `move_group` melden.
4. `RosBridge.start()` – ohne ROS nur Warnung.
5. `_load(urdf_path)` – Modell lesen, prüfen (Hinweisdialog bei Problemen), Tabellen und Slider bauen, `robot_description` senden.
6. Takte `_tick` und `_record_tick` starten.

`move_group` wird **nicht** automatisch gestartet; das geschieht über „Neu starten“ im Parameterfenster (6.3).

## 6.2 Modelländerung

Jede Änderung am Modell läuft durch `_commit(beschreibung, mutate)`:

1. Abbruch, wenn gesperrt (`_model_locked`).
2. Tiefe Kopie von `xml_root`, `mutate(kopie)` (Funktionen aus `urdf_modell`).
3. `validate_root(kopie)` – bei Fehlern Dialog, nichts wird übernommen.
4. Alten Stand in den Verlauf (max. `MAX_HISTORY` = 25), Kopie übernehmen, `_rebuild_from_root()` (Tabellen, Slider, TF, Baum).
5. `_moveit_stale()` – laufender `move_group` gilt als veraltet; Planen ist gesperrt, bis neu gestartet wird.

Gespeichert wird erst mit „Speichern“ (atomar, mit Sicherung).

## 6.3 `move_group` starten

„Neu starten“ → `_params_restart()` → `_params_apply()` (Felder lesen, prüfen, `MoveItConfig` ersetzen) → `_moveit_start()`: SRDF bauen, `ros.reset_moveit("")`, sperren („Neustart move_group“), `MoveItProcess.start()`, `ros.reset_moveit(knotenname)`. Ab dann wartet `_moveit_watch()` im `_tick` bis `moveit_ready` (Freigabe), bis der Prozess stirbt (letzte Logzeilen ins Meldungsfenster) oder bis `MOVEIT_WAIT_S` = 25 s vergangen sind.

## 6.4 Einzelauftrag: planen und fahren

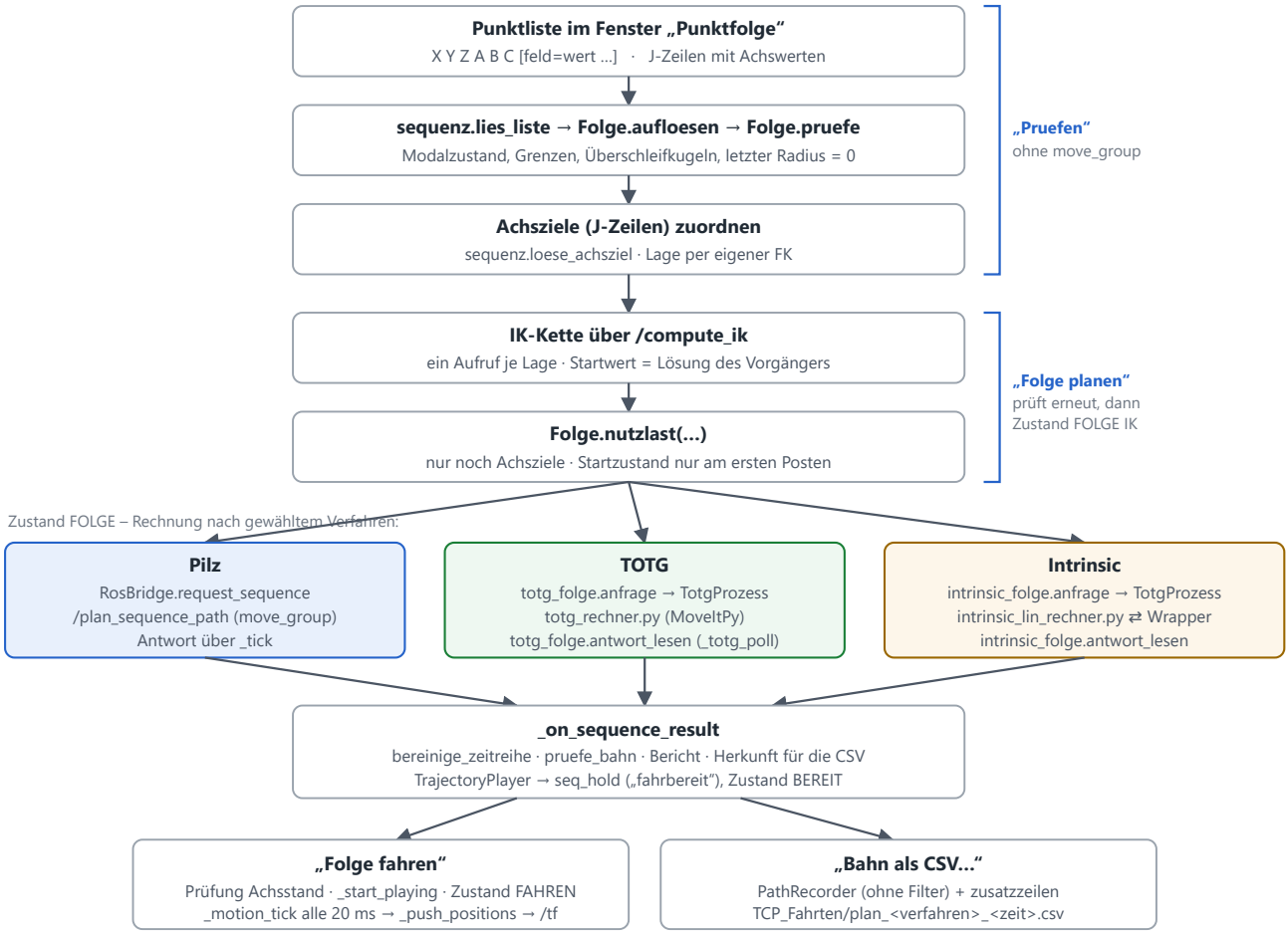
1. „Planen“ (`_drive_plan`): Prüfungen (nicht beschäftigt, kein STOP, `moveit_ready`, nicht veraltet).
2. Kartesisch: `_ik_request()` → Zustand IK → `/compute_ik` → `_on_ik_result` schreibt die Lösung in die Achszielfelder und ruft `_plan_joint_goal()`.
3. `_plan_joint_goal()`: Zielfelder lesen und auf URDF-Grenzen begrenzen, Auftrag mit Startzustand = aktueller Stand → Zustand PLANEN → `request_plan`.
4. `_on_plan_result`: bei Erfolg `TrajectoryPlayer` als `plan_hold` halten (Zustand BEREIT) – oder, falls der Halte-Taster noch gedrückt ist, sofort fahren (`_hold_launch`).
5. „Fahrt starten“ (`_drive_start`): Achsstand muss exakt dem beim Planen entsprechen (sonst Plan verwerfen), dann `_start_playing()` → Zustand FAHREN → `_motion_tick` alle 20 ms → `_push_positions()` → `/tf`.
6. Ende: „Ziel erreicht“ oder nach Bremsrampe „angehalten“; Meldung mit Bahnzeit, Wall-Clock-Zeit („Wanduhr“) und Abweichung.

Jede Änderung eines Sollwerts, der Fahrparameter oder des Zielraums verwirft den gehaltenen Plan (`_drop_plan`).

## 6.5 Joggen

- **Kartesisch** (Fahrfenster, Zielraum „Kartesisch“): ein Klick verschiebt die geführte Sollpose um `JOG_STEP_M` / `JOG_STEP_DEG`; nach `JOG_HOLD_DELAY_MS` wird daraus eine Dauerfahrt mit `JOG_VEL_*`. Jeder Schritt ist eine `/compute_ik`-Anfrage; der nächste Schritt folgt erst auf die Antwort. Die Achsen werden direkt gestellt, ohne Plan. `_jog_report` zeigt die Verstärkung (Achsweg je Bahnweg) und die Auslastung der schnellsten Achse (Warnung ab 75 %, Rückfall unter 65 %).
- **Achse** (Zielraum „Achsraum“): stellt die Achse direkt im 50-ms-Takt, ohne `move_group`; hält am URDF-Anschlag an.

## 6.6 Punktfolge




*Ablauf einer Punktfolge*

1. `_seq_read()`: Liste lesen, Achsziele auflösen (`loese_achsziel`, Lage per FK), Folge auflösen und prüfen. Bei Pilz/TOTG sind überlappende Überschleifkugeln ein Fehler, bei Intrinsic ein Hinweis.
2. `_seq_plan()`: Vorprüfungen je Verfahren (`sequence_ready`, `_seq_totg_bereit`, `_seq_intrinsic_bereit`), Warnung bei Pilz mit Ruckig und Radius > 0, dann Zustand FOLGE IK.
3. `_seq_ik_next()` / `_on_seq_ik_result()`: je kartesischem Punkt eine IK-Anfrage mit der Lösung des Vorgängers als Startwert; Achsziele werden ohne Anfrage übernommen. Achssprünge über `SEQ_JUMP_DEG` = 90° werden gelb gemeldet.
4. `_seq_send()`: `Folge.nutzlast()` mit Achszielen, dann je Verfahren `ros.request_sequence`, `_totg_send` oder `_intrinsic_send` → Zustand FOLGE.
5. `_on_sequence_result()`: Zeitreihe bereinigen, Bericht schreiben, Herkunft für die CSV bauen (`_seq_herkunft_bauen`: Verfahren, Parameter, Eckwerte), `TrajectoryPlayer` als `seq_hold` halten.

6. „Folge fahren“ (`_seq_drive`): zwei Prüfungen – Achsstand gleich dem beim Planen, und erster Stützpunkt innerhalb `FOLGE_START_TOL` – dann `_start_playing()`.

### 6.7 Aufzeichnen und Exportieren

„Aufzeichnen“ startet den `PathRecorder` mit dem aktuellen Achs- und Framesatz und sperrt die Drehkonvention. `_record_tick` rechnet alle 40 ms die FK, nimmt TCP-Lage, Achswerte und alle Frames und bietet sie an; Stillstand wird gefiltert. Ein Modellwechsel, ein anderer aktiver TCP oder ein geänderter Achssatz verwerfen die Aufzeichnung. „Export…“ schreibt `write_csv`. „Bahn als CSV…“ im Folgenfenster schreibt dagegen die *geplante* Bahn: ein eigener Recorder ohne Filter tastet jeden Stützpunkt ab, dazu kommen die Zusatzzeilen des Verfahrens.

## 7 Erweitern und Ändern

### 7.1 Konstanten ändern

- Immer im Modul ändern, das die Konstante definiert. `urdf_gui.py` hält nur eine Kopie der Bindung (`from bahn import IPO_TICK_MS`); eine Änderung dort wirkt in den anderen Modulen nicht.
- Wer ein Basismodul ändert, erhöht dort `STAND` und trägt den neuen Wert in `MODUL_STAENDE` in `urdf_gui.py` ein. So bleibt jede Messreihe einer Kombination von Dateien zuordenbar.
- Einige Grenzen spiegeln Vorgaben von MoveIt, Pilz-Patch oder Intrinsic Core (in Kommentaren „gespiegelt“/`[NACHGEBILDET]`). Bei Abweichung gilt das Original.

### 7.2 Ein weiteres Folgeverfahren anbinden

Das Muster von TOTG und Intrinsic lässt sich wiederholen:

1. **Reines Modul** `<name>_folge.py` (kein Tk/ROS): `pruefe_parameter(…)`, `anfrage(nutzlast, …) → (auftrag, befunde)`, `antwort_lesen(text, gelenke)` im Format von `totg_folge.antwort_lesen`, `zusatzzeilen(herkunft)`, `STAND`, `FRIST_S`, Konstante `VERFAHREN_<NAME>`.
2. **Rechner** `<name>_rechner.py` mit `AUFTRAG.json ANTWORT.json` und `--pruefen`, atomare Antwort, `os._exit`.
3. **GUI**: zweites `TotgProzess(rechner=…)` anlegen; Eintrag in der Verfahrensauswahl (`_seq_build_verfahren`) mit eigenen Feldern und deren Zuordnung in `seq_totg_widgets`; `_seq_<name>_bereit()` (Vorprüfung vor der IK-Kette), `_<name>_send(payload)`, `_<name>_poll()` (in `_tick_rumpf` eintragen); Abbruch in `_brake`, Aufräumen in `_on_close`; Bericht in `_on_sequence_result`; Zusatzzeilen in `_seq_export`.
4. **Tests** nach dem Vorbild von `test_intrinsic_folge.py` (Stub für Grenzen und Wrapper) und `test_gui_intrinsic.py` (GUI-Methoden aus dem Quelltext herauslösen).

### 7.3 Regeln für Threads, Takte und Fehler

- Kein ROS-Aufruf und kein blockierendes Warten im GUI-Thread.
- Neue Takte nach dem Muster `try: … except: melden finally: after(…)`.
- Neue Callbacks im Executor immer über `_guard(name, funktion)`.
- Neue Aufträge an `RosBridge` immer mit `tag` aus `_job_open()` und Prüfung mit `_job_mine()`; jede Anfrage braucht einen Timeout (`timeout`).
- Bedienelemente nur in `_update_locks()` aktivieren oder sperren.
- Bei jeder neuen Art, den Achsstand zu ändern, über `_push_positions()` gehen – sonst verfallen gehaltene Pläne nicht und die Anzeige im Fahrfenster bleibt stehen.

# 8 Tests

Alle aktuellen Tests laufen ohne ROS, ohne Tk und ohne Bildschirm (`unittest`):

```
python3 test_fassung_2_3.py            # 20 Tests: Aufräumarbeiten 2.3 (s. unten)
python3 test_intrinsic_folge.py        # 33 Tests: Auftrag Intrinsic (Stub-Wrapper)
python3 test_intrinsic_lin_rechner.py  # 35 Tests: Ablauf des Rechners, Stub-Wrapper
python3 test_gui_intrinsic.py          # 10 Tests: GUI-Methoden, aus urdf_gui.py herausgelöst
python3 test_moveit_konfig_kdl.py      #  6 Tests: KDL-Parameter in der Parameterdatei
python3 test_sequenz_kugeln.py         #  4 Tests: kugeln_pruefen
python3 mutation_intrinsic_folge.py    # Mutationsproben gegen test_intrinsic_folge.py
```

Stand der Dokumentation: alle 108 Tests grün. `test_gui_intrinsic.py` liest `urdf_gui.py` als Quelltext (`ast`) und hängt einzelne Methoden an ein Stub-Objekt – die GUI selbst lässt sich ohne Tk/ROS nicht starten. `test_fassung_2_3.py` prüft `Folge.nutzlast` (nur Achsziele), `RosBridge._send_sequence` mit Mocks für `moveit_msgs`, `PLAN_FIELDS`, die Pfade und Umgebungsvariablen in `einstellungen.py`, `MODUL_STAENDE` gegen die Module und die Karte BEWEGUNG (als Quelltext). Ältere Tests (u. a. `test_sequenz.py`, `test_totg_folge.py`, `test_konvention.py`) liegen in `archiv/Alte_Tests/` und gehören zu früheren Fassungen.

Von den Mutationsproben melden zwei „PROBE KAPUTT“: sie suchen eine Zeile in `intrinsic_folge.pruefe_parameter`, die seit dem Umbau auf eine Schleife nicht mehr so dasteht. Die übrigen 36 werden erkannt.

# 9 Bekannte Einschränkungen und offene Punkte

Beim Durchsehen des Codes aufgefallen; keine davon verhindert den normalen Betrieb. Was davon die Bedienung betrifft und wie es weitergehen kann, steht in Teil II, Abschnitt 13.

1. **CIRC in der Punktliste nicht nutzbar.** `sequenz._lies_felder` kennt kein Feld für den Hilfspunkt; eine Zeile mit `fahrart=CIRC` scheitert deshalb immer an der Prüfung („CIRC braucht einen Hilfspunkt“). Im Fahrfenster lehnt Pilz CIRC mangels Zwischenpunkt ebenfalls ab.
2. **Reiter „Kartesisch“, Taste „Go“** prüft und protokolliert nur; sie bewegt nichts (Weg über „Ins Fahrfenster“).
3. `sequenz.MODAL_START` (PTP, 10 %, 10 %, 0 m) gilt nur, wenn `Folge` ohne `modal` gebaut wird. In der GUI kommt der Anfangszustand aus dem Parameterfenster (Standard 100 %).
4. **Intrinsic-Variante:** kein Jerk-Limit, keine Kollisionsprüfung, ein Verrundungsradius für die ganze Kurve, keine Grenze für Drehgeschwindigkeit des Tips.
5. **Zwei Mutationsproben veraltet** (siehe Abschnitt 8).

## 9.1 Behoben in Fassung 2.3

| Punkt | Änderung | Module (Stand) |
|---|---|---|
| Lage-Zweig der Nutzlast passte nicht zu `RosBridge` | `Folge.nutzlast` lehnt eine Lage ohne Achswerte ab (`tcp`-Parameter entfällt); `_send_sequence` nimmt nur Achsziele und meldet einen Posten ohne `goal` als Fehler | `sequenz`, `ros_bruecke` (2) |
| Karte BEWEGUNG zeigte andere Werte als wirksam | Abgleich direkt nach dem Fensteraufbau, Vorgaben aus `CARTESIAN_FIELDS`, Jerk entfernt, Überschrift „(Neustart noetig)“; die Verzögerung folgt weiter der Beschleunigung, jetzt mit Meldung | `urdf_gui` |
| `bahn_viewer4.py` in der Exportmeldung | `BAHN_VIEWER = "bahn_viewer3.py"` in `einstellungen`, Meldung mit vollem Pfad | `einstellungen`, `totg_prozess` (2) |
| Veralteter Kopfkommentar | Abschnitt 2.2 berichtigt, Abschnitt 2.3 ergänzt | `urdf_gui` |

| Punkt | Änderung | Module (Stand) |
|---|---|---|
| Felder ohne Wirkung im Reiter „Fahrt“ | `goal_position_tol`, `max_step`, `jump_threshold` entfernt; `goal_orientation_tol` heißt „Zieltoleranz Achsen“ | `moveit_konfig` (2) |
| Absolute Pfade | Pfade relativ zum Modulordner, Umgebungsvariablen; Beschreibungen der Launch-Argumente | `einstellungen` (2), `display_launch4.py` |
| Aufräumbedarf | `.gitignore`; `Zone.Identifier`, `--wrapper` gelöscht; `__pycache__` aus dem Index; Archiv und alte Arbeitsdateien nach `archiv/` | – |

# Anhang A · Konfigurationsreferenz

Konstanten, die man beim Einrichten oder Experimentieren anfasst. Geändert wird immer im genannten Modul (siehe 7.1).

| Modul | Konstante | Standard | Bedeutung |
|---|---|---|---|
| `einstellungen` | `URDF_PATH` | `bmKinematic.urdf` neben dem Modul (`$URDF_GUI_URDF`) | URDF ohne Startargument |
| | `BACKUP_DIR` | `URDF_Speicher/` neben dem Modul (`$URDF_GUI_SICHERUNG`) | Sicherungen (absolut oder relativ zur URDF) |
| | `BAHN_DIR` | `TCP_Fahrten/` neben dem Modul (`$URDF_GUI_BAHNEN`) | Vorschlagsordner CSV |
| | `BAHN_VIEWER` | `bahn_viewer3.py` | Bahnansicht, die nach dem Export genannt wird |
| | `TF_RATE_HZ` | 30 | TF-Takt |
| | `FOOTER_TICK_MS` | 50 | Fußzeilentakt |
| | `MAX_LOG_LINES`, `MAX_HISTORY` | 600, 25 | Meldungszeilen, Rückgängig-Tiefe |
| | `SLIDER_REFRESH_MS`, `SLIDER_STEPS` | 60, 1000 | Tabellen-Neubau beim Ziehen, Slider-Stufen |
| | `FREE_ROTATION_DEG`, `FREE_TRAVEL_M` | 180, 1.0 | Slider-Spanne ohne URDF-Anschlag |
| | `RECORD_RATE_HZ` | 25 | Takt des Recorders |
| | `RECORD_MIN_DIST_M`, `RECORD_MIN_ANGLE_DEG`, `RECORD_MIN_JOINT_DEG` | 0.0005, 0.5, 0.5 | Stillstandsfilter |
| | `RECORD_MAX_POINTS` | 20000 | Puffergröße (≈ 13 min) |
| `bahn` | `IPO_TICK_MS` | 20 | Interpolationstakt |
| | `BRAKE_TIME_S` | 0.40 | Bremsrampe |
| | `HOLD_WATCH_MS` | 100 | Watchdog Halte-Taster |
| | `JOB_RATE_HZ`, `SPIN_ERROR_LIMIT`, `TF_STALL_MS` | 20, 20, 1000 | Executor, Fehlergrenze, TF-Watchdog |
| | `IK_TIMEOUT_MARGIN_S` | 5.0 | Zuschlag auf den IK-Timeout |

| Modul | Konstante | Standard | Bedeutung |
|---|---|---|---|
| | `JOG_STEP_M`, `JOG_STEP_DEG` | 0.01, 0.1 | Jog-Schritt je Klick |
| | `JOG_VEL_M_S`, `JOG_VEL_DEG_S` | 0.02, 0.2 | Jog-Geschwindigkeit beim Halten |
| | `JOG_HOLD_DELAY_MS`, `JOG_MAX_DT_S`, `JOG_TICK_MS` | 300, 0.25, 50 | Jog-Zeiten |
| | `JOG_WARN_SHARE`, `JOG_CLEAR_SHARE` | 0.75, 0.65 | Hysterese Achsauslastung |
| | `FOLGE_START_TOL` | 1e-4 rad | Startprüfung Folge |
| `moveit_konfig` | `MOVEIT_GROUP` | `manipulator` | Planning Group |
| | `MOVEIT_WAIT_S`, `PLAN_TIMEOUT_MARGIN_S` | 25, 10 | Hochlauf, Zuschlag auf den Planungs-Timeout |
| | `ACC_FROM_VEL`, `JERK_FROM_ACC`, `DEFAULT_JOINT_VELOCITY` | 2, 10, 1.0 | Ableitung fehlender Joint Limits |
| | `PLAN_FIELDS` | 100 %, 100 %, 5 s, 1, 0.1°, 0 m | je Auftrag (Teil II, Abschnitt 5.1) |
| | `CARTESIAN_FIELDS` | 1.0 m/s, 2.25, −5.0 m/s², 1.57 rad/s | Cartesian Limits (Pilz) |
| | `PLANNER_FIELDS` | `sampling_time` 0.1 s (1e-4 … 1) | nur mit Pilz-Patch |
| | `KINEMATICS_FIELDS`, `KDL_FIELDS` | 0.05 s, 0.005 rad; 500, 1e-5, 1.0 | IK-Solver |
| `sequenz` | `ACHSZIEL_KENNUNG`, `FRIST_ZUSCHLAG_S`, `ORT_GLEICH_M` | `J`, 5 s, 1e-9 m | Punktliste |
| `totg_folge` | `PATH_TOLERANCE_STANDARD`, `RESAMPLE_DT_STANDARD`, `MIN_ANGLE_CHANGE` | 0.1, 0.01 s, 0.001 | TOTG |
| | `FRIST_S`, `UMKEHR_TOLERANZ` | 90 s, 1e-5 | Timeout, Umkehrerkennung |
| `intrinsic_folge` | `WRAPPER_STANDARD` | `~/intrinsic-core/bazel-bin/…` | Wrapper ohne Umgebungsvariable |
| | `AST_FOLGEN`, `UEBERSCHLEIFEN_RAD` | `True`, 0.2 | IK-Ast folgen, Überschleifen im Joint Space |
| | `RUNDUNG_ROT_STANDARD`, `RUNDUNG_TRANS_OHNE_RADIUS` | 0.1 rad, 0.001 m | Verrundung |
| | `GELENK_ABTASTUNG_STANDARD` | 0.05 rad | Resampling im Joint Space |
| | `TCP_V_MIN/MAX`, `TCP_A_MIN/MAX` | 1e-4…10 m/s, 1e-3…100 m/s² | Eingabegrenzen v_bahn, a_bahn |
| | `FRIST_S` | 300 s | Timeout des Rechners |
| `intrinsic_lin_rechner` | `IK_TOLERANZ_M`, `IK_TOLERANZ_RAD` | 1e-3, 1e-2 | FK-Prüfung der IK |
| | `MAX_IK_SPRUNG_RAD`, `START_TOLERANZ_RAD`, `ZIEL_TOLERANZ_RAD` | π/4, 1e-3, 1e-3 | Prüfungen nach Intrinsic |

| Modul | Konstante | Standard | Bedeutung |
|---|---|---|---|
| | `STANDARD` | 0.002 m, 0.005 rad, 0.05 rad, 0.2 rad | Abtastung/Überschleifen wie Intrinsic |
| `anzeige` | `ENTARTUNG_WARNUNG_DEG` | 1.0 | Warnung Winkelsingularität |
| `konvention` | `STANDARD` | `ZYX` | voreingestellte Drehkonvention |
| `bahn_viewer3` | `FAST_MAX_POINTS`, `POINT_BUDGET`, `SPEED_BUCKETS` | 1500, 4000, 24 | Zeichenaufwand |

# Teil II · Bedienerdokumentation

> Für alle, die mit der Oberfläche Bahnen planen, abfahren, aufzeichnen und vergleichen. Programmierkenntnisse sind nicht nötig; vorausgesetzt werden Grundbegriffe der Robotik wie Achse, TCP, PTP- und LIN-Bewegung; Abkürzungen und Fachbegriffe erklärt das Verzeichnis vor Teil I. Die Beschriftungen der Oberfläche sind ohne Umlaute geschrieben (z. B. „Uebernehmen“); diese Anleitung zitiert sie genau so, wie sie auf dem Bildschirm stehen.

## 1 Überblick

Die Oberfläche bewegt ein **virtuelles** Robotermodell. Sie stellt die Achswerte selbst, rechnet daraus die Lage aller Links und schickt sie an RViz2, das den Roboter zeichnet. Ein echter Roboter wird nicht angesteuert.

Was man damit tun kann:

- ein Robotermodell (URDF) laden, ansehen und bearbeiten,
- Achsen direkt stellen oder joggen,
- Einzelbewegungen planen und abfahren (PTP, LIN),
- Punktfolgen mit drei Verfahren planen und vergleichen (Pilz, TOTG, Intrinsic),
- die Bahn des TCP (Tool Center Point) aufzeichnen und als CSV speichern,
- Bahnen im Programm **Bahnansicht** in 3D ansehen und vergleichen.

| Fenster | geöffnet über | Zweck |
|---|---|---|
| Hauptfenster „URDF Viewer“ | Programmstart | Modellübersicht, Achsen stellen, Aufzeichnen |
| „Bewegungsparameter“ | „Parameter…“ | Planungswerte, Grenzen, `move_group` starten |
| „Fahren“ | „Fahren…“ | Ziel vorgeben, planen, fahren, joggen |
| „Punktfolge“ | „Punktfolge…“ | Punktlisten prüfen, planen, fahren, exportieren |
| „URDF bearbeiten“ | „Bearbeiten…“ | Modell ändern, speichern |
| „Bahnansicht“ | eigenes Programm `bahn_viewer3.py` | CSV-Bahnen in 3D ansehen |

## 2 Installation

Die Schritte bauen einen Rechner so auf, wie die Software getestet ist. Auf einem fertig eingerichteten Rechner geht es direkt mit Abschnitt 3 weiter.

| Baustein | getesteter Stand | gebraucht für |
|---|---|---|
| Windows 11 mit WSL2 | Windows 11 Pro 23H2 | nur unter Windows; ein nativ installiertes Ubuntu geht ebenso |
| Ubuntu | 24.04 LTS „Noble“ | alles |
| ROS 2 | Jazzy ( `ros-jazzy-desktop` ) | Oberfläche mit ROS, RViz2 |
| MoveIt 2 mit Pilz | 2.12.4 | Planen, IK, Punktfolgen |

| Baustein | getesteter Stand | gebraucht für |
|---|---|---|
| `moveit_py` | 2.12.4 | Verfahren TOTG und Intrinsic |
| Python | 3.12 mit `tkinter`, PyYAML, NumPy, Pillow | Oberfläche, Rechner, Bildexport der Bahnansicht |
| Intrinsic Core | Release `20260922.0` | nur Verfahren Intrinsic |
| Bazel (über Bazelisk) | 8.8.0 | nur Bau des Intrinsic-Wrappers |

Rechner: x86-64 mit 16 GB Arbeitsspeicher und rund 30 GB freiem Platz, davon etwa 19 GB allein für den Bazel-Cache von Intrinsic Core.

## 2.1 Ubuntu 24.04 unter Windows (WSL2)

Entfällt bei einem nativ installierten Ubuntu 24.04. In einer PowerShell mit Administratorrechten:

```
wsl --install -d Ubuntu-24.04
```

Danach Windows neu starten und „Ubuntu 24.04" aus dem Startmenü öffnen; beim ersten Start fragt Ubuntu nach Benutzername und Passwort. Dann das System aktualisieren:

```
sudo apt update && sudo apt upgrade -y
```

- Die Fenster der Linux-Programme (Oberfläche, RViz2) zeigt Windows 11 über WSLg ohne weitere Einrichtung an.
- RViz2 braucht unter WSL Software-Rendering. Das Launch-Skript setzt es selbst (`software_gl:=true`, Standard).
- Im Windows-Explorer sind die Linux-Dateien unter `\\wsl.localhost\<Distribution>\home\<benutzer>\` erreichbar, z. B. `\\wsl.localhost\Ubuntu-24.04\home\<benutzer>\Display`.

## 2.2 ROS 2 Jazzy und MoveIt 2

Gekürzt nach der offiziellen Anleitung „Ubuntu (deb packages)" für Jazzy auf docs.ros.org; bei Abweichungen gilt die offizielle Anleitung.

```
# UTF-8-Locale und Paketquelle "universe"
sudo apt update && sudo apt install -y locales software-properties-common curl
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
sudo add-apt-repository -y universe

# Paketquelle von ROS 2 eintragen
export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-infrastructure/ros-apt-
source/releases/latest | grep -F "tag_name" | awk -F\" '{print $4}')
curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/ros-apt-
source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VERSION}.$(.
/etc/os-release && echo ${UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"
sudo dpkg -i /tmp/ros2-apt-source.deb
```

```
# ROS 2, MoveIt 2 (mit Pilz und KDL) und moveit_py
sudo apt update && sudo apt upgrade -y
sudo apt install -y ros-jazzy-desktop ros-dev-tools ros-jazzy-moveit ros-jazzy-moveit-py

# optional: TRAC-IK als weiterer IK-Solver (Reiter "Kinematik")
sudo apt install -y ros-jazzy-trac-ik-kinematics-plugin

# Python-Pakete der Oberfläche
sudo apt install -y python3-tk python3-yaml python3-numpy python3-pil

# ROS in jedem neuen Terminal laden
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Auf dem Entwicklungsrechner liegt über den apt-Paketen zusätzlich ein Quellcode-Build von MoveIt 2.12.4 in `~/ws_moveit` (Overlay); `moveit_py` und der gepatchte Pilz-Planner kommen von dort. Die apt-Pakete haben dieselbe Version.

**Optional: Pilz mit einstellbarer Abtastzeit.** Das Feld „Abtastzeit“ (Abschnitt 5.3) wirkt nur mit einem gepatchten Pilz-Planner; sonst rechnet Pilz fest mit 0,1 s. Dafür wird nur das Pilz-Paket aus dem Quellcode gebaut, in einem eigenen Workspace über den apt-Paketen:

```
mkdir -p ~/ws_moveit/src && cd ~/ws_moveit/src
git clone -b jazzy --filter=blob:none --sparse https://github.com/moveit/moveit2.git moveit2_src
cd moveit2_src && git sparse-checkout set moveit_planners/pilz_industrial_motion_planner
python3 <pfad>/patch_pilz_sampling_time_1.py \
    ~/ws_moveit/src/moveit2_src/moveit_planners/pilz_industrial_motion_planner
cd ~/ws_moveit
colcon build --packages-select pilz_industrial_motion_planner --cmake-args -DCMAKE_BUILD_TYPE=Release
echo "source ~/ws_moveit/install/setup.bash" >> ~/.bashrc
```

Ob der Patch geladen ist, zeigt das Log von `move_group` (`/tmp/urdf_gui_moveit_*/move_group.log`) mit der Zeile „PILZ-PATCH aktiv“. Das Skript ist gegen Pilz 2.12.4 geprüft; mit `--pruefen` stellt es fest, ob der Patch angewendet ist, mit `--zurueck` entfernt es ihn.

## 2.3 Intrinsic Core und Wrapper

Nur für das Verfahren Intrinsic. Gebraucht wird nicht die Laufzeitumgebung von Intrinsic Core (k3s, `inctl`, OMTS aus deren Getting-Started-Anleitung), sondern nur ein Teil der Bibliotheken für Bahnkurven und TOPP-RA. Daraus baut Bazel ein einzelnes Programm, den `intrinsic_toppra_wrapper`.

```
# Intrinsic Core in genau diesem Release
sudo apt install -y git curl
cd ~
git clone --branch 20260922.0 https://github.com/intrinsic-ai/intrinsic-core.git

# Bazelisk als "bazel"; holt die Bazel-Version aus .bazelversion (8.8.0)
curl -L https://github.com/bazelbuild/bazelisk/releases/latest/download/bazelisk-linux-amd64 -o
/tmp/bazelisk
sudo install -m 755 /tmp/bazelisk /usr/local/bin/bazel

# Quellen des Wrappers in den Baum von Intrinsic Core legen
mkdir -p ~/intrinsic-core/intrinsic_toppra_wrapper
cp BUILD intrinsic_toppra_wrapper.cc json_klein.h ~/intrinsic-core/intrinsic_toppra_wrapper/

# bauen; --jobs=4 begrenzt den Speicherbedarf
cd ~/intrinsic-core
bazel build --jobs=4 //intrinsic_toppra_wrapper:intrinsic_toppra_wrapper

# prüfen
bazel-bin/intrinsic_toppra_wrapper/intrinsic_toppra_wrapper --pruefen
```

Erwartete Ausgabe der Prüfung:

```
intrinsic_toppra_wrapper Stand 3, intrinsic-core 20260922.0
ok
```

- Beim ersten Bau lädt Bazel einen eigenen Compiler (LLVM) und alle Fremdbibliotheken herunter. Das dauert lange und belegt in `~/.cache/bazel` rund 19 GB; spätere Bauten gehen schnell.
- Die Anleitung von Intrinsic Core setzt für die vollständige Laufzeitumgebung Ubuntu 26.04 voraus. Der Wrapper allein baut und läuft unter Ubuntu 24.04 (getestet).
- Intrinsic Core (Apache 2.0) bleibt unverändert; der Wrapper ist eigener Code und nutzt dessen Bibliotheken nur als Abhängigkeiten.
- Liegt der Wrapper an anderer Stelle als `~/intrinsic-core/bazel-bin/intrinsic_toppra_wrapper/intrinsic_toppra_wrapper`, den Pfad mit `export INTRINSIC_WRAPPER=…` in `~/.bashrc` oder beim Start mit `intrinsic_wrapper:=…` angeben.
- Nach einer Änderung an `intrinsic_toppra_wrapper.cc` denselben `bazel build` wiederholen. Die Oberfläche verlangt mindestens Wrapper-Stand 3.

> **Noch nicht im Repository:** Die drei Quelldateien des Wrappers (`BUILD`, `intrinsic_toppra_wrapper.cc`, `json_klein.h`) und das Skript `patch_pilz_sampling_time_1.py` liegen bisher nur auf dem Entwicklungsrechner (`~/intrinsic-core/intrinsic_toppra_wrapper/` bzw. außerhalb des Projektordners). Für eine neue Installation müssen sie von dort kopiert werden.

## 2.4 Projekt einrichten

Den Projektordner, ein Git-Repository ohne eingerichtetes Remote, nach `~/Display` kopieren. Der Ort ist frei wählbar: Modell, Sicherungen und Bahnen liegen standardmäßig neben den Skripten, und das Launch-Skript findet alles relativ zu sich selbst (Abschnitt 12).

```
cd ~/Display
git status        # vollständig und ohne fremde Änderungen?
```

## 2.5 Installation prüfen

```
cd ~/Display
python3 test_fassung_2_3.py                 # Tests ohne ROS, am Ende "OK"
python3 totg_rechner.py --pruefen           # "moveit_core 2.12.4 <pfad>" und "ok"
python3 intrinsic_lin_rechner.py --pruefen  # ebenso
ros2 launch display_launch4.py              # Oberfläche und RViz2
```

- Der Pfad hinter `moveit_core` zeigt, welches MoveIt geladen wird: `/opt/ros/jazzy/…` (apt) oder `~/ws_moveit/install/…` (Overlay).
- Eine Zeile „double free or corruption" nach dem „ok" entsteht beim Beenden von MoveItPy; das Ergebnis der Prüfung gilt trotzdem.
- In der Oberfläche **Parameter…** → **Neu starten**; nach einigen Sekunden steht in der Fußzeile `move_group bereit`. Dann im Fenster „Punktfolge" das Beispiel nacheinander mit Pilz, TOTG und Intrinsic planen.

## 2.6 Ordnerstruktur

Auf dem Rechner:

```
~/                                      Home-Verzeichnis in Ubuntu
├── Display/                            dieses Projekt (Ort frei wählbar)
├── intrinsic-core/                     Intrinsic Core 20260922.0
│   ├── intrinsic_toppra_wrapper/       Quellen des Wrappers (eigener Code)
│   └── bazel-bin/intrinsic_toppra_wrapper/
│       └── intrinsic_toppra_wrapper    gebautes Programm (Standardpfad)
├── ws_moveit/                          optional: Overlay mit gepatchtem Pilz
│   └── src/  build/  install/
└── .cache/bazel/                       Bazel-Cache (rund 19 GB)

/opt/ros/jazzy/                         ROS 2 Jazzy und MoveIt 2 (apt)
/tmp/urdf_gui_moveit_*/                 Arbeitsordner von move_group (zur Laufzeit)
/tmp/urdf_gui_totg_*/                   Arbeitsordner der Rechner (zur Laufzeit)
```

Im Projektordner:

```
Display/
├── urdf_gui.py                     Oberfläche (Hauptprogramm)
├── display_launch4.py              Launch: Oberfläche und RViz2
├── display2.rviz                   RViz-Konfiguration
├── bmKinematic.urdf                Standardmodell
├── bahn_viewer3.py                 Bahnansicht, eigenständig
├── einstellungen.py  stil.py  matrix.py  anzeige.py  urdf_modell.py
│   kinematik.py  moveit_konfig.py  bahn.py  totg_prozess.py  ros_bruecke.py
│                                   Basismodule
├── konvention.py  sequenz.py  totg_folge.py  intrinsic_folge.py
│                                   Rechenmodule ohne Tk und ROS
├── totg_rechner.py  intrinsic_lin_rechner.py
│                                   Rechner, laufen als Kindprozesse
├── intrinsic_lin_auftrag.py        Joint Limits lesen; Hilfsskript
├── test_*.py  mutation_intrinsic_folge.py
│                                   Tests
├── patch_lin_rechner_mimic_3.py    früherer Patch des LIN-Rechners
├── README.md  .gitignore
├── docs/                           diese Dokumentation (Markdown, Bilder, PDF)
├── TCP_Fahrten/                    Bahnen als CSV (bahn_*, plan_*)
├── URDF_Speicher/                  Sicherungen der URDF
└── archiv/                         frühere Fassungen, nur zum Nachlesen
```

Welche Datei wozu dient, beschreibt Teil I, Abschnitt 3.2; die Ablageorte zur Laufzeit stehen in Abschnitt 12.

# 3 Starten und Beenden

In einem Terminal mit ROS-Umgebung:

```
source /opt/ros/jazzy/setup.bash
cd ~/Display
ros2 launch display_launch4.py
```

Es öffnen sich das Hauptfenster und nach etwa 1,5 s RViz2. Varianten:

| Aufruf | Wirkung |
|---|---|
| `ros2 launch display_launch4.py urdf:=/pfad/robot.urdf` | anderes Modell laden |
| `… rviz:=false` | ohne RViz2 |
| `… software_gl:=false` | RViz mit Grafikkarte (unter WSL nicht verwenden) |
| `… intrinsic_wrapper:=/pfad/zum/intrinsic_toppra_wrapper` | anderer Ort des Intrinsic-Wrappers |
| `python3 urdf_gui.py [URDF]` | nur die Oberfläche, ohne Launch (ohne Angabe: `bmKinematic.urdf`) |

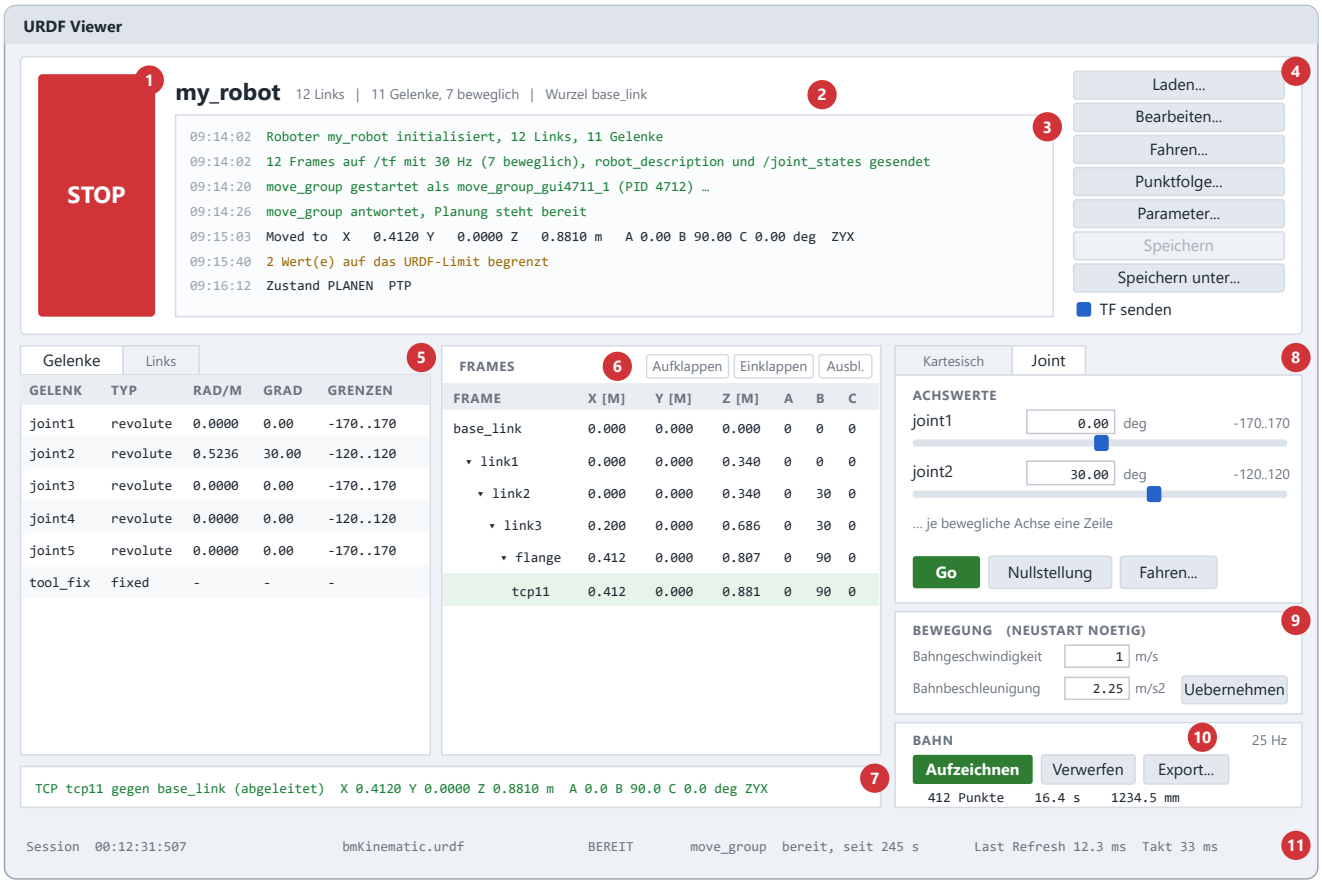
**Beenden:** das Hauptfenster schließen. Gibt es ungespeicherte Änderungen am Modell, fragt das Programm nach. Mit dem Hauptfenster enden auch RViz2, `move_group` und alle Rechenprozesse.

## 3.1 Erste Schritte

1. Programm starten. Im Kopf steht der Robotername, darunter die Zahl der Links und Gelenke. Hinweise zur URDF erscheinen gelb im Meldungsfenster.
2. **Parameter…** öffnen und **Neu starten** drücken. Damit startet `move_group`, der Planner von MoveIt. Warten, bis in der Fußzeile `move_group bereit` steht (einige Sekunden).
3. **Fahren…** öffnen, Zielwerte eintragen, **Planen**, dann **Fahrt starten**.
4. Wer die Bahn aufheben will, drückt vorher im Hauptfenster unter BAHN auf **Aufzeichnen** und danach **Export…**.
5. Die CSV mit `python3 bahn_viewer3.py <datei>` ansehen.

> Ohne gestarteten `move_group` lässt sich nur direkt stellen (Slider, Achsfelder, Achs-Joggen). Planen, kartesisches Joggen und Punktfolgen brauchen ihn.

# 4 Hauptfenster




*Hauptfenster (schematisch)*

1. **STOP** – sperrt sofort alle Sollwerte. Eine laufende Fahrt wird über die Bremsrampe (0,4 s) angehalten, eine laufende Planung verworfen. Die Taste heißt danach **FREI** (grün); ein Druck darauf hebt STOP wieder auf. Die Anzeige in RViz läuft weiter.
2. **Robotername und Kennzahlen** – Zahl der Links und Gelenke, davon beweglich, und der Root-Link (in der Oberfläche „Wurzel“).
3. **Meldungsfenster** – alle Ereignisse mit Uhrzeit: schwarz Information, grün Erfolg, gelb Warnung, rot Fehler. Es hält die letzten 600 Zeilen.
4. **Tasten** – „Laden…“ (andere URDF), „Bearbeiten…“, „Fahren…“, „Punktfolge…“, „Parameter…“, „Speichern“ (nur aktiv bei ungespeicherten Änderungen; der Fenstertitel trägt dann ein `*`), „Speichern unter…“ und der Haken **TF senden**. Ohne Haken sendet die Oberfläche keine Lagen mehr; RViz verliert die Frames nach etwa 10 s.
5. **Reiter „Gelenke“ und „Links“** – Tabelle aller Gelenke mit Typ, aktuellem Wert (rad/m und grad/mm), Grenzen und Kette (Eltern > Kind); bzw. aller Links mit Masse, Schwerpunkt, Form und Farbe.
6. **FRAMES** – Lage jedes Links gegenüber dem Root-Link: X, Y, Z in m und die Winkel A, B, C in Grad in der gewählten Drehkonvention. Die Zeile des TCP ist grün hinterlegt, rot hinterlegt sind Links ohne Verbindung zum Root-Link. „Aufklappen“/„Einklappen“ öffnet bzw. schließt alle Zweige, „Ausblenden“ klappt die ganze Karte weg (ein schmaler Streifen „FRAMES“ holt sie zurück).
7. **TCP-Zeile** – Lage des aktiven TCP, mit Bezug, Herkunft („gewaehlt“ oder „abgeleitet“) und Drehkonvention. Gelb heißt: B steht weniger als 1° neben einer Stellung, in der A und C nicht mehr eindeutig sind (Winkelsingularität der Darstellung, nicht des Roboters). Gibt es mehrere Endeffektoren und ist keiner gewählt, steht hier „Mehrere Endeffektoren, keiner gewaehlt“.
8. **Sollwerte, Reiter „Joint“ und „Kartesisch“** – siehe 4.1 und 4.2.

9. **BEWEGUNG (Neustart noetig)** – Bahngeschwindigkeit und Bahnbeschleunigung für LIN und CIRC. Die Felder zeigen von Anfang an die wirksamen Werte, dieselben wie im Parameterfenster unter „Kartesisch". „Uebernehmen" schreibt sie in die Parameter; sie wirken nach „Neu starten" von `move_group`. Die Bahnverzögerung wird dabei auf denselben Betrag gesetzt – das Meldungsfenster sagt es.
10. **BAHN** – Aufzeichnung der TCP-Bahn, siehe 4.4.
11. **Fußzeile** – Sitzungsdauer, geladene Datei, Zustand der Ablaufsteuerung (z. B. `BEREIT`, `PLANEN`, `FAHREN`), Zustand von `move_group`, Alter der letzten TF-Ausgabe und gemessener TF-Takt.

## 4.1 Achsen direkt stellen (Reiter „Joint")

Je bewegliche Achse gibt es ein Zahlenfeld (Grad bei Drehachsen, Meter bei Schubachsen), den Bereich aus der URDF und einen Slider.

- **Slider ziehen** – die Achse folgt sofort, RViz zeigt es unmittelbar. Pfeiltasten auf einem Slider gehen um genau eine Stufe. Das Mausrad verstellt bewusst nichts.
- **Wert eintippen** – übernommen wird mit Return, beim Verlassen des Feldes oder mit **Go**. Werte jenseits der URDF-Grenze werden auf die Grenze gesetzt.
- **Nullstellung** – alle Achsen auf 0.
- Gekoppelte Achsen (`<mimic>` in der URDF) haben keinen Slider; dort steht die Rechenvorschrift, z. B. `gekoppelt: -1 * joint6`.

Farben der Eingabefelder (gilt in allen Fenstern):

| Farbe | Bedeutung |
|---|---|
| weiß | Wert übernommen |
| gelb | auf die Grenze begrenzt (bzw. bei Overrides auf 100 %) |
| rot | nicht lesbar oder unzulässig, nichts übernommen |

Direkt gestellte Werte brauchen keinen `move_group` und keinen Plan. Sie verwerfen aber einen gehaltenen Plan (der Startpunkt stimmt nicht mehr).

## 4.2 Reiter „Kartesisch"

Sechs Felder für eine Ziellage im Basissystem: X, Y, Z in m, A, B, C in Grad.

- **TCP uebernehmen** – trägt die aktuelle TCP-Lage ein.
- **Go** – prüft nur, ob die Eingabe lesbar ist, und schreibt sie ins Meldungsfenster. Es bewegt sich nichts.
- **Ins Fahrfenster** – öffnet das Fahrfenster im Zielraum „Kartesisch (IK)" mit diesen Werten. Dort wird geplant und gefahren.

## 4.3 Drehkonvention

Die Winkel A, B, C werden je nach Einstellung (Parameterfenster, Reiter „Anzeige") nach ZYX, ZYZ oder ZXZ gebildet. A ist immer die erste, C die letzte Drehung. Standard ist ZYX (wie URDF und viele Industriesteuerungen: A = Gieren um Z, B = Nicken um Y, C = Rollen um X). Die Einstellung wirkt sofort auf alle Anzeigen und Eingabefelder, aber nicht auf die URDF: die Bearbeiten-Fenster rechnen immer in ZYX (beschriftet „rpy … ZYX").

## 4.4 Bahn aufzeichnen (Karte BAHN)

- **Aufzeichnen** startet die Aufzeichnung mit 25 Hz; die Taste heißt dann **Anhalten** (rot). Erneutes Aufzeichnen setzt eine angehaltene Aufnahme fort. Aufgezeichnet wird jede Bewegung – Slider, Joggen, Fahrten.
- Ein Punkt wird nur abgelegt, wenn sich etwas bewegt hat (0,5 mm Weg, 0,5° Drehung oder 0,5° bzw. 0,5 mm an einer Achse).

- **Verwerfen** löscht die Punkte (eine laufende Aufzeichnung sammelt weiter).
- **Export…** speichert eine CSV, vorgeschlagen in `TCP_Fahrten/` als `bahn_<Datum>_<Uhrzeit>.csv`.
- Die Zeile darunter zeigt Punkte, Dauer und gefahrenen Weg; „(Puffer voll)" heißt, dass vorne die ältesten Punkte herausfallen (ab 20 000 Punkten, etwa 13 Minuten).

Während einer Aufzeichnung lässt sich die Drehkonvention nicht umschalten. Ein Modellwechsel oder ein anderer aktiver TCP verwirft die Aufzeichnung.

# 5 Fenster „Bewegungsparameter"

Geöffnet über **Parameter…**. Hier wird `move_group` gestartet und konfiguriert. Die Werte zerfallen in zwei Gruppen:

- **Reiter „Fahrt"** – wirkt beim nächsten Auftrag, ohne Neustart.
- **alle anderen MoveIt-Reiter** (Überschrift „Neustart noetig") – wirken erst, wenn `move_group` neu gestartet wird.
- **Reiter „Anzeige"** – wirkt sofort und betrifft nur die Darstellung.

| Taste | Wirkung |
|---|---|
| **Uebernehmen** | Werte prüfen und übernehmen. Läuft `move_group` und hat sich ein neustartpflichtiger Wert geändert, gilt er danach als „veraltet". |
| **Neu starten** | Übernehmen und `move_group` mit dem aktuellen Modell neu starten. Die Oberfläche ist gesperrt, bis er antwortet (höchstens 25 s). |
| **Stoppen** | `move_group` beenden. |
| **Fremde beenden** | `move_group`-Prozesse beenden, die nicht von dieser Oberfläche stammen (nach Rückfrage). |

Die Statuszeile im Fuß zeigt denselben Text wie die Fußzeile des Hauptfensters:

| Anzeige | Bedeutung |
|---|---|
| `move_group aus` | nicht gestartet – Planen ist nicht möglich |
| `move_group startet, wartet auf Antwort` | Hochlauf |
| `move_group bereit, seit N s` | betriebsbereit |
| `move_group keine Antwort, seit N s` | Prozess läuft, Services fehlen |
| `move_group laeuft, aber veraltet` | Modell oder Parameter geändert – **Neu starten** |
| `move_group beendet / fehler` | Prozess ist weg; die letzten Zeilen seiner Ausgabe stehen im Meldungsfenster |

## 5.1 Reiter „Fahrt" (pro Auftrag)

| Feld | Einheit | Standard | Wirkung |
|---|---|---|---|
| Fahrart | – | PTP | PTP, LIN oder CIRC für das Fahrfenster; Anfangswert der Punktliste |
| Override Geschwindigkeit | % | 100 | Skalierung der Achs- bzw. Bahngeschwindigkeit (0 ist nicht erlaubt – MoveIt läse daraus 100 %) |
| Override Beschleunigung | % | 100 | Skalierung der Beschleunigung |
| Planungszeit | s | 5.0 | Zeit je Planungsversuch |
| Versuche | – | 1 | Planungsversuche; auch Faktor für den IK-Timeout |
| Zieltoleranz Achsen | deg | 0.1 | wie genau jede Achse ihr Ziel erreichen muss |

| Feld | Einheit | Standard | Wirkung |
|---|---|---|---|
| Ueberschleifradius | m | 0.0 | Anfangswert `radius` der Punktliste |

Fahrart und die beiden Overrides stehen zusätzlich im Fahrfenster; beide Fenster zeigen immer denselben wirksamen Wert.

## 5.2 Reiter „Achsgrenzen“ (Joint Limits, Neustart nötig)

Je bewegliche Achse vier Felder: Geschwindigkeit `v`, Beschleunigung `a`, Verzögerung `dec` (negativ einzutragen) und Jerk `jerk`. **Ein leeres Feld schaltet die Grenze aus.** „Aus URDF fuellen“ setzt die Werte zurück: Geschwindigkeit aus der URDF, Beschleunigung = 2 × Geschwindigkeit, Jerk = 10 × Beschleunigung. Pilz rechnet mit der strengsten Achse für alle; TOTG und Intrinsic brauchen eine Beschleunigungsgrenze je Achse.

## 5.3 Reiter „Kartesisch“ (Neustart nötig)

| Feld | Einheit | Standard |
|---|---|---|
| Bahngeschwindigkeit | m/s | 1.000 |
| Bahnbeschleunigung | m/s² | 2.250 |
| Bahnverzoegerung | m/s² | −5.000 |
| Drehgeschwindigkeit | rad/s | 1.570 |
| Abtastzeit | s | 0.100 |

Diese Cartesian Limits gelten für LIN und CIRC mit Pilz. Die Abtastzeit wirkt nur mit einem gepatchten Pilz-Planner (Abschnitt 2.2, sonst fest 0,1 s); halbieren verdoppelt etwa die Planungszeit einer LIN-Bahn.

## 5.4 Reiter „Kinematik“ (Neustart nötig)

| Feld | Standard | Hinweis |
|---|---|---|
| Solver | KDL | KDL, TRAC-IK oder LMA |
| Zeitgrenze je Aufruf | 0.050 s | bei LIN mal Zahl der Stützpunkte – häufigste Ursache für lange Planungszeiten |
| Suchschrittweite | 0.005 rad | |
| Planungsgruppe (Planning Group) | `manipulator` | muss zur erzeugten SRDF passen |
| Tip-Link | leer | TCP für IK und Planung; leer = einziger Leaf-Link des Modells |
| KDL: Iterationen je Versuch / Konvergenzschwelle / Gewicht Drehung : Lage | 500 / 1e-5 / 1.0 | nur mit KDL |
| Nur Lage (`position_only_ik`) | aus | für LIN falsch – die Drehung wird dann nicht eingehalten |

## 5.5 Reiter „Pipeline“ (Neustart nötig)

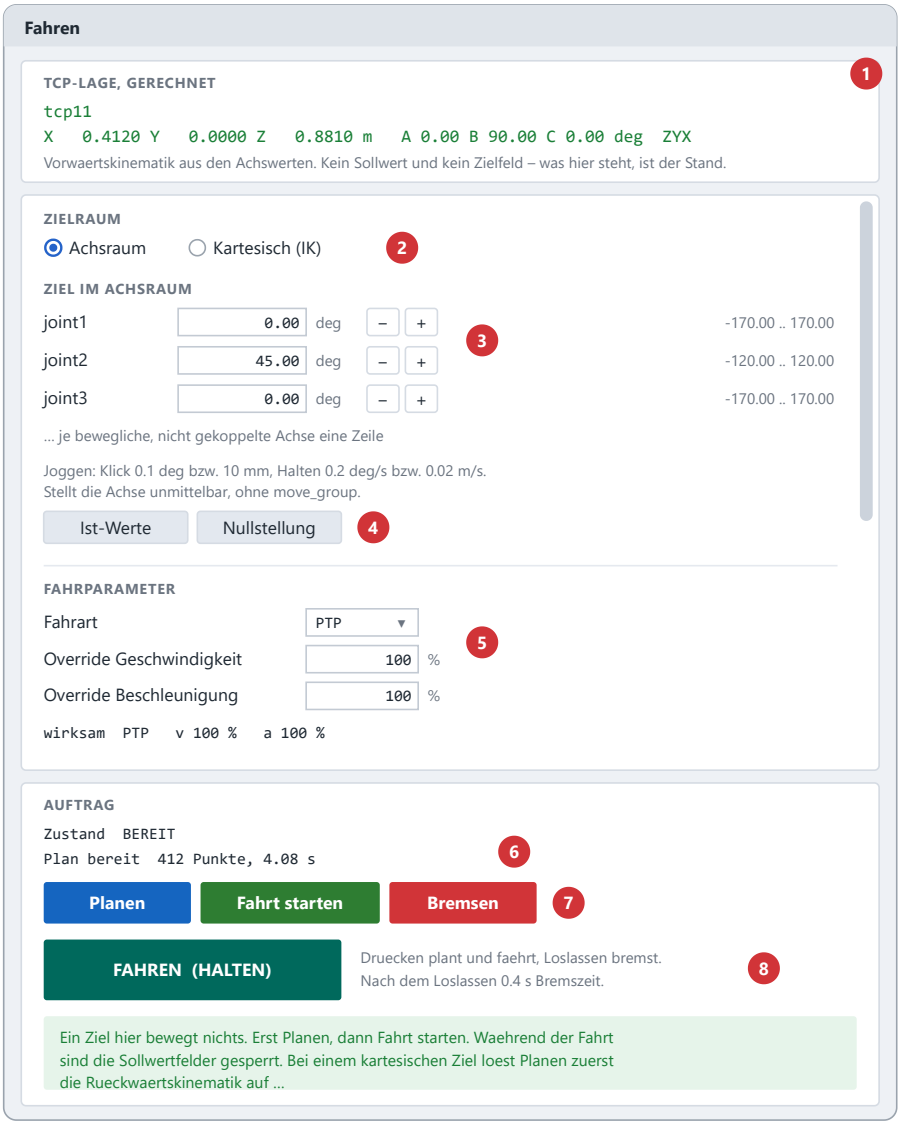
| Haken | Standard | Bedeutung |
|---|---|---|
| Ruckig-Glaettung anhaengen | an | begrenzt den Jerk der Pilz-Bahn. **Für Punktfolgen mit Radius > 0 ausschalten** – beides zusammen scheitert in der Regel (Fehler 99999). |

| Haken | Standard | Bedeutung |
|---|---|---|
| Bahn auf Kollisionen pruefen | an | prüft Start und Bahn auf Kollisionen der Links untereinander (ausgenommen die Paare der ACM in der SRDF). Grobe Ersatzkörper melden dabei Treffer, die es mechanisch nicht gibt; aus heißt „nicht geprüft“. |
| Ausfuehrung durch move_group zulassen | aus | nur einschalten, wenn ros2_control läuft. Die Oberfläche fährt selbst. |

### 5.6 Reiter „Anzeige“

Auswahl der Drehkonvention ZYX, ZYZ oder ZXZ (siehe 4.3), mit einer Tabelle der Drehachsen und der Stellungen, an denen die Darstellung entartet (ZYX bei B = ±90°, ZYZ/ZXZ bei B = 0° und 180°).

## 6 Fenster „Fahren“




*Fenster „Fahren“ (schematisch)*

Das Fahrfenster trennt drei Handlungen: **Ziel setzen** (es bewegt sich nichts), **Planen** (move_group rechnet, das Ergebnis wird gehalten) und **Fahrt starten** (die gehaltene Bahn wird abgefahren).

1. **TCP-LAGE, GERECHNET** – die aktuelle Lage des TCP, auch während der Fahrt laufend aktualisiert. Das ist der Ist-Stand, kein Zielfeld.
2. **ZIELRAUM** – „Achsraum“ (Zielwerte je Achse) oder „Kartesisch (IK)“ (Ziellage X … C). Beide Feldsätze bleiben erhalten; das Umschalten verwirft nur einen gehaltenen Plan.
3. **Zielzeile** – Zielwert, Einheit, Jogtasten **−/+** und der Bereich aus der URDF. Return in einem Zielfeld startet „Planen“.
4. **Ist-Werte** / **Nullstellung** übernehmen den aktuellen Stand bzw. Null als Ziel (im kartesischen Zielraum: **TCP uebernehmen**).
5. **FAHRPARAMETER** – Fahrart und die beiden Overrides (siehe 6.4).
6. **AUFTRAG** – Zustand und Plan: „kein Plan“, „Plan bereit N Punkte, T s“, „wird gefahren“.
7. **Planen** (blau), **Fahrt starten** (grün, nur mit gehaltenem Plan), **Bremsen** (rot).
8. **FAHREN (HALTEN)** – der Halte-Taster (siehe 6.3).

## 6.1 Planen und fahren

1. Zielraum wählen und Ziel eintragen.
2. **Planen**. Im Achsraum geht der Auftrag direkt an den Planner; im kartesischen Zielraum wird zuerst die Inverse Kinematics (IK) gerechnet (Zustand `IK`), die Lösung erscheint danach als Achsziel, und es wird automatisch weitergeplant (Zustand `PLANEN`).
3. War die Planung erfolgreich, steht „Plan bereit …“ da. Im Meldungsfenster stehen Planungszeit, Zahl der Stützpunkte und Bahndauer.
4. **Fahrt starten**. Die Bahn wird im 20-ms-Takt abgefahren; Sollwertfelder, Modell und Parameter sind währenddessen gesperrt.
5. **Bremsen** (oder STOP) führt die Bahn über 0,4 s auf Stillstand – auf der Bahn, ohne Sprung.

Nach der Fahrt meldet das Programm Takte, Bahnzeit, Wall-Clock-Zeit (in der Meldung „Wanduhr“) und deren Abweichung.

## 6.2 Wann ein Plan verfällt

Ein gehaltener Plan gilt nur für den Achsstand, aus dem er gerechnet wurde. Er wird verworfen (gelbe Meldung „Plan verworfen: …“), sobald

- eine Achse gestellt oder gejoggt wird,
- Fahrart oder ein Override geändert wird,
- der Zielraum umgeschaltet wird,
- ein anderes Modell geladen oder das Modell geändert wird.

Dann einfach erneut **Planen**.

## 6.3 Halte-Taster „FAHREN (HALTEN)“

Der Halte-Taster verbindet Planen und Fahren in einem Griff:

- **Drücken und halten** – es wird geplant und, sobald die Bahn da ist, sofort gefahren.
- **Loslassen** – bremst (0,4 s). Erst danach nimmt der Taster wieder an.
- Rutscht der Zeiger von der Taste, verliert das Fahrfenster den Fokus oder wird es geschlossen, wird ebenfalls gebremst. Ein Watchdog prüft das alle 100 ms.
- Die Bahn endet an ihrem Ziel; weiteres Halten fährt nicht weiter. Für den nächsten Auftrag loslassen und neu drücken.

### 6.4 Fahrparameter

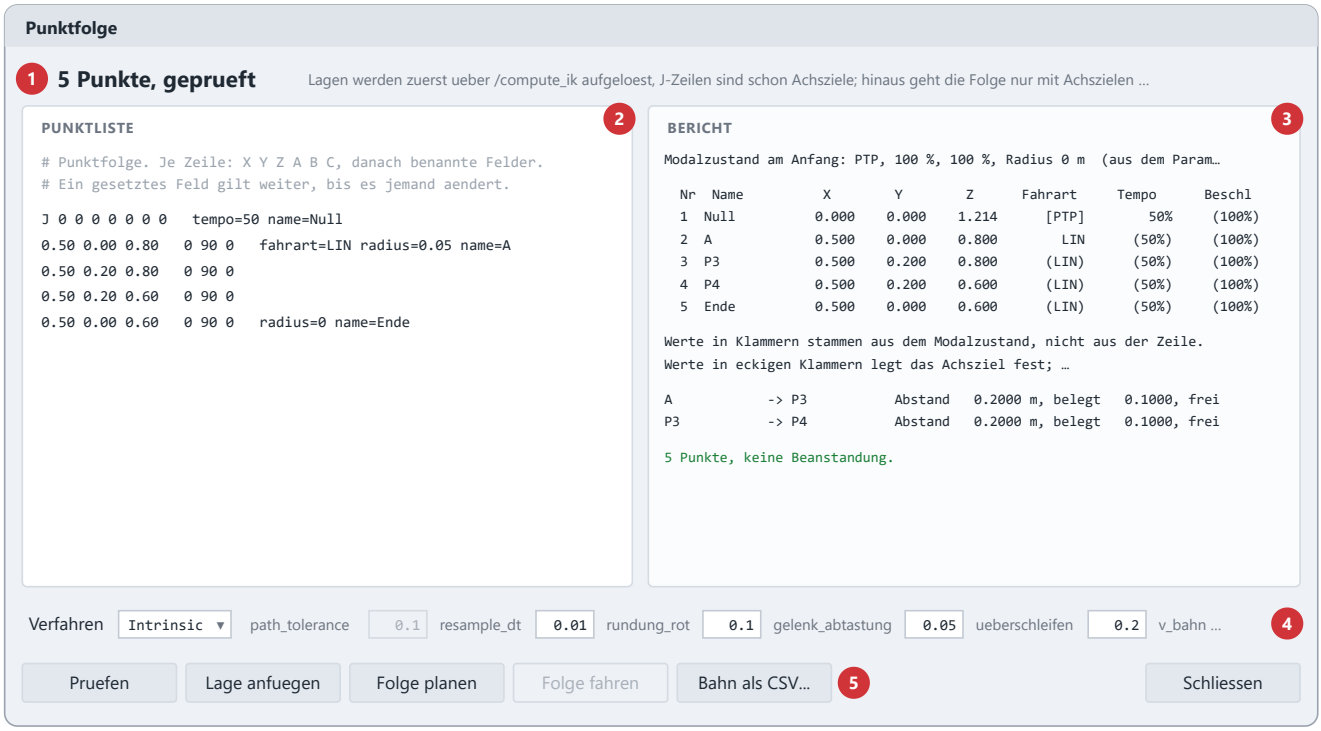
**Fahrart** (PTP, LIN, CIRC) gilt mit der Auswahl. Die **Overrides** gelten, sobald die Eingabe fertig ist – mit Return oder beim Verlassen des Feldes; Escape holt den wirksamen Wert zurück. Die Zeile darunter zeigt den wirksamen Stand, z. B. `wirksam PTP v 100 % a 100 %`, und meldet „Eingabe offen“, solange ein Feld abweicht. Werte über 100 % werden auf 100 % begrenzt, 0 oder negative Werte abgelehnt. CIRC braucht einen Zwischenpunkt, den das Fahrfenster nicht kennt – für Fahrten aus diesem Fenster PTP oder LIN nehmen.

### 6.5 Joggen

| | Achsraum | Kartesisch (IK) |
|---|---|---|
| Klick | 0,1° bzw. 10 mm | 10 mm bzw. 0,1° |
| Halten (ab 0,3 s) | 0,2 °/s bzw. 0,02 m/s | 0,02 m/s bzw. 0,2 °/s |
| braucht `move_group` | nein | ja (jeder Schritt eine IK-Anfrage) |
| Grenze | hält am URDF-Anschlag an | stoppt, wenn keine IK-Lösung gefunden wird |

Beim kartesischen Joggen zeigt eine Zeile unter den Feldern die **Verstärkung** (wie viel Achsweg ein Millimeter Bahn kostet) und die Auslastung der schnellsten Achse. Gelb ab 75 % der Achsgrenze heißt Singularitätsnähe; rot über 100 % heißt, dass eine Gerade durch diese Stellung von Pilz abgelehnt würde.

## 7 Fenster „Punktfolge“




*Fenster „Punktfolge“ (schematisch)*

1. **Statuszeile** – z. B. „5 Punkte, geprueft“, „Pilz: 412 Punkte, 4.08 s, 1 Teilbahn(en) fahrbereit“ oder der Grund eines Fehlers.
2. **PUNKTLISTE** – Eingabefeld, beim ersten Öffnen mit einem Beispiel.
3. **BERICHT** – Ergebnis von Prüfung, IK-Kette und Planung.
4. **Verfahren** – Pilz, TOTG oder Intrinsic, dazu die Felder des Verfahrens (nicht zutreffende Felder sind grau).

5. **Tasten** – „Pruefen“, „Lage anfuegen“, „Folge planen“, „Folge fahren“, „Bahn als CSV…“, „Schliessen“.

## 7.1 Punktliste schreiben

Eine Zeile je Punkt. Leerzeilen und alles hinter `#` werden ignoriert.

**Lage-Zeile:** `X Y Z A B C`, danach benannte Felder.

```
0.50 0.00 0.80   0 90 0   fahrart=LIN tempo=80 radius=0.05 name=Anfahrt
```

X, Y, Z in Metern im Basissystem, A, B, C in Grad in der eingestellten Drehkonvention.

| Feld | Werte | Bedeutung |
|---|---|---|
| `fahrart=` | `PTP`, `LIN` (`CIRC` siehe unten) | Bewegungsart zu diesem Punkt |
| `tempo=` | 0 < x ≤ 100 | Geschwindigkeits-Override in % |
| `beschleunigung=` | 0 < x ≤ 100 | Beschleunigungs-Override in % |
| `radius=` | ≥ 0 m | Überschleifradius an diesem Punkt; 0 = Genauhalt |
| `name=` | Text ohne Leerzeichen | Name im Bericht (sonst P1, P2, …) |

**Die Felder sind modal:** ein gesetzter Wert gilt für alle folgenden Zeilen, bis er neu gesetzt wird – wie `F1000` im G-Code. Den Anfangszustand liefert das Parameterfenster (Fahrart, Overrides, Überschleifradius). Der Bericht zeigt geerbte Werte in runden Klammern.

**Achsziel-Zeile (J):** `J` und je bewegliche Achse ein Wert (Grad, bei Schubachsen Meter), in der Reihenfolge des Fahrfensters, gekoppelte Achsen ausgenommen.

```
J 0 0 0 0 0 0   tempo=50 name=Null
```

Ein Achsziel fährt immer PTP ohne Überschleifen; `fahrart=` und `radius=` sind dort nicht erlaubt, `tempo=`, `beschleunigung=` und `name=` schon. Achsziele eignen sich als wiederholbarer Startpunkt, weil eine Lage bei sieben Achsen mehrdeutig ist. Werte außerhalb der URDF-Grenzen sind ein Fehler (sie werden nicht begrenzt).

Regeln, die „Pruefen“ meldet:

- Der **letzte Punkt braucht** `radius=0`.
- Bei Pilz und TOTG dürfen sich die **Überschleifkugeln** zweier Nachbarpunkte nicht überlappen: Radius(i) + Radius(i+1) < Abstand. Der Bericht zeigt je Paar Abstand, belegten und freien Platz. Bei Intrinsic ist das nur ein Hinweis.
- `CIRC` ist in der Punktliste derzeit nicht nutzbar: es gibt kein Feld für den nötigen Hilfspunkt, die Prüfung meldet deshalb immer einen Fehler.

**Lage anfuegen** hängt die aktuelle TCP-Lage als neue Zeile mit `radius=0` an („Teachen“: Roboter hinstellen, Lage übernehmen).

## 7.2 Verfahren wählen

| | Pilz | TOTG | Intrinsic |
|---|---|---|---|
| Rechnung | `move_group`, Service `/plan_sequence_path` | MoveIt-TOTG im Hilfsprozess `totg_rechner.py` | Intrinsic Core (Kurve, TOPP-RA) mit MoveIt-IK im Hilfsprozess |
| Fahrarten | PTP, LIN | Gerade im Joint Space; LIN wirkt nicht (Warnung) | LIN und PTP |

| | Pilz | TOTG | Intrinsic |
|---|---|---|---|
| Radius 0 | Genauhalt | Genauhalt | Genauhalt |
| Radius > 0 | Überschleifkugel mit diesem Radius | verschleifen mit `path_tolerance`, der Wert wird nicht gelesen | zwischen LIN-Zeilen: Eckenverrundung mit **einem** Radius für alle; sonst Überschleifen im Joint Space |
| Tempo/Beschleunigung | je Punkt | ein Wert für die ganze Folge | ein Wert für die ganze Folge |
| Voraussetzung | Ruckig aus bei Radius > 0 | `move_group` aus dem Parameterfenster gestartet und nicht veraltet | wie TOTG, zusätzlich Wrapper und Tip-Link |
| Rechenzeit | Sekundenbruchteile | einige Sekunden (MoveIt startet) | länger; Timeout 5 min |

Felder in der Verfahrenszeile:

| Feld | Einheit | Standard | gilt für | Bedeutung |
|---|---|---|---|---|
| `path_tolerance` | rad | 0.1 | TOTG | Verschleifen an den Ecken im Joint Space (> 0 … 10) |
| `resample_dt` | s | 0.01 | TOTG, Intrinsic | Abstand der ausgegebenen Stützpunkte (1e-4 … 1) |
| `rundung_rot` | rad | 0.1 | Intrinsic | Verrundung der Drehung an den Ecken |
| `gelenk_abtastung` | rad | 0.05 | Intrinsic | Abstand der Stützpunkte im Joint Space; zu grob → kurze Ecken verschwinden |
| `ueberschleifen` | rad | 0.2 | Intrinsic | Überschleifen im Joint Space (Standard wie Intrinsic) |
| `v_bahn` | m/s | leer | Intrinsic | Obergrenze der TCP-Geschwindigkeit (leer = keine), mal Override |
| `a_bahn` | m/s² | leer | Intrinsic | Obergrenze der TCP-Beschleunigung (leer = keine), mal Override |

Umschalten des Verfahrens verwirft die Fahrbereitschaft einer geplanten Folge, nicht deren Ergebnis; die CSV lässt sich weiter schreiben.

## 7.3 Prüfen, planen, fahren

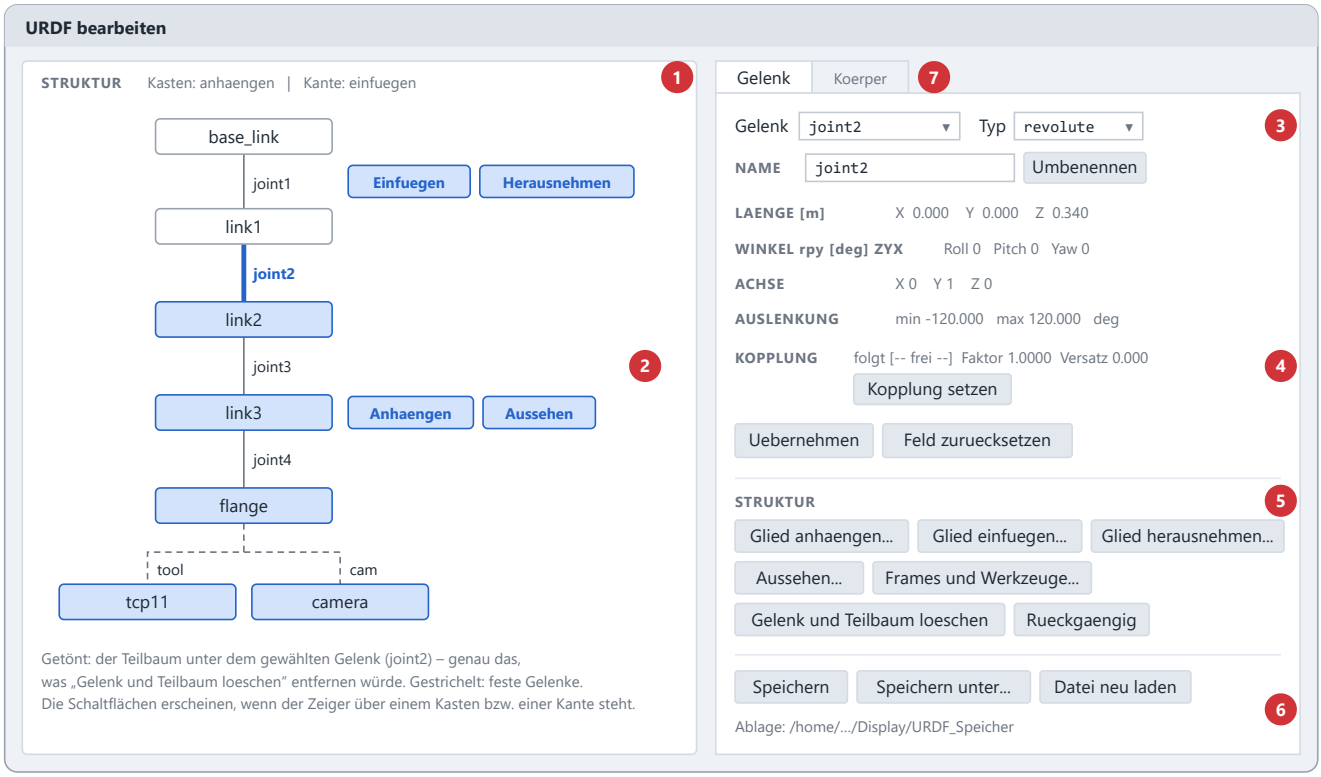
1. **Pruefen** – liest die Liste und prüft Format, Grenzen und Radien, ohne `move_group`. Alle Fehler erscheinen auf einmal im Bericht.
2. **Folge planen** – prüft erneut und löst dann jeden Lagepunkt über die Inverse Kinematics (IK) auf, jeweils mit der Lösung des Vorgängers als Startwert (Zustand `FOLGE IK`). Je Punkt erscheinen im Bericht die Achswerte und der größte Achsschritt; über 90° wird die Zeile gelb (Wechsel des IK-Asts). Danach rechnet das gewählte Verfahren (Zustand `FOLGE`).
3. Im Bericht stehen danach Planungszeit, Zahl der Teilbahnen, Stützpunkte, Bahndauer und je Zwischenpunkt die **Ecken**: kleinster Abstand der geplanten TCP-Bahn zum Punkt (mm), Geschwindigkeit dort (mm/s) und Zeitpunkt. Am Ende steht „Fahrbereit“.
4. **Folge fahren** – fährt die Folge wie eine Einzelfahrt ab. Das geht nur aus genau dem Achsstand, in dem geplant wurde; wer dieselbe Folge zweimal fahren will, plant sie zweimal.
5. **Bahn als CSV…** – schreibt die *geplante* Bahn (alle Stützpunkte, ungefiltert) mit Verfahren, Parametern und Eckwerten im Dateikopf, als `plan_<verfahren>_<Datum>_<Uhrzeit>.csv`.

Bremsen oder STOP während der IK-Kette oder der Rechnung bricht die Planung ab.

### 7.4 Bericht lesen

| Darstellung | Bedeutung |
|---|---|
| `(10%)` | Wert aus dem Modalzustand geerbt, nicht in der Zeile gesetzt |
| `[PTP], [0 m]` | vom Achsziel festgelegt |
| gelbe IK-Zeile | Achssprung über 90° zwischen zwei Punkten |
| „1 Teilbahn(en)“ bei Pilz | alle Übergänge verschmolzen, kein Zwischenhalt |
| „N Teilbahnen“ bei Pilz | an N − 1 Übergängen wird angehalten |
| „Abschnitt(e)“ bei TOTG/Intrinsic | Stücke zwischen Genauhalten und Umkehrpunkten |
| „Ecken … Abstand … v …“ | gemessene Bahnabweichung und Geschwindigkeit an jedem Zwischenpunkt |

## 8 Fenster „URDF bearbeiten“




*Fenster „URDF bearbeiten“ (schematisch)*

1. **STRUKTUR** – der Gelenkbaum: Kästen sind Links, Linien Gelenke (gestrichelt = fest). Ein Klick wählt ein Gelenk (ein Klick auf einen Kasten wählt das Gelenk, das zu ihm führt). Der Teilbaum darunter wird blau getönt – genau das, was „Gelenk und Teilbaum loeschen“ entfernen würde.
2. **Schaltflächen im Baum** – steht der Zeiger über einem Kasten, erscheinen „Anhaengen“ und „Aussehen“; über einer Linie „Einfuegen“ und „Herausnehmen“.
3. **Gelenk** und **Typ** – Auswahl des Gelenks und Typ (revolute, continuous, prismatic, fixed). Der Typ schaltet nur die Felder um; geschrieben wird mit „Uebernehmen“.
4. **Gelenkfelder** – NAME mit eigener Taste „Umbenennen“; LAENGE (Ursprung X, Y, Z in m); WINKEL (Roll, Pitch, Yaw in Grad, immer ZYX); ACHSE; AUSLENKUNG (min/max in Grad bzw. m); KOPPLUNG („folgt“, Faktor, Versatz, Taste „Kopplung setzen“ – wird als `<mimic>` in die URDF geschrieben). „Feld zuruecksetzen“ lädt die Werte des Gelenks neu.

5. **STRUKTUR-Tasten** – „Glied anhaengen…“, „Glied einfuegen…“, „Glied herausnehmen…“, „Aussehen…“, „Frames und Werkzeuge…“, „Gelenk und Teilbaum loeschen“, „Rueckgaengig“ (bis 25 Schritte).
6. **Datei** – „Speichern“, „Speichern unter…“, „Datei neu laden“ (verwirft ungespeicherte Änderungen). Darunter der Sicherungsordner.
7. **Reiter „Koerper“** – Aussehen eines Links (siehe 8.2).

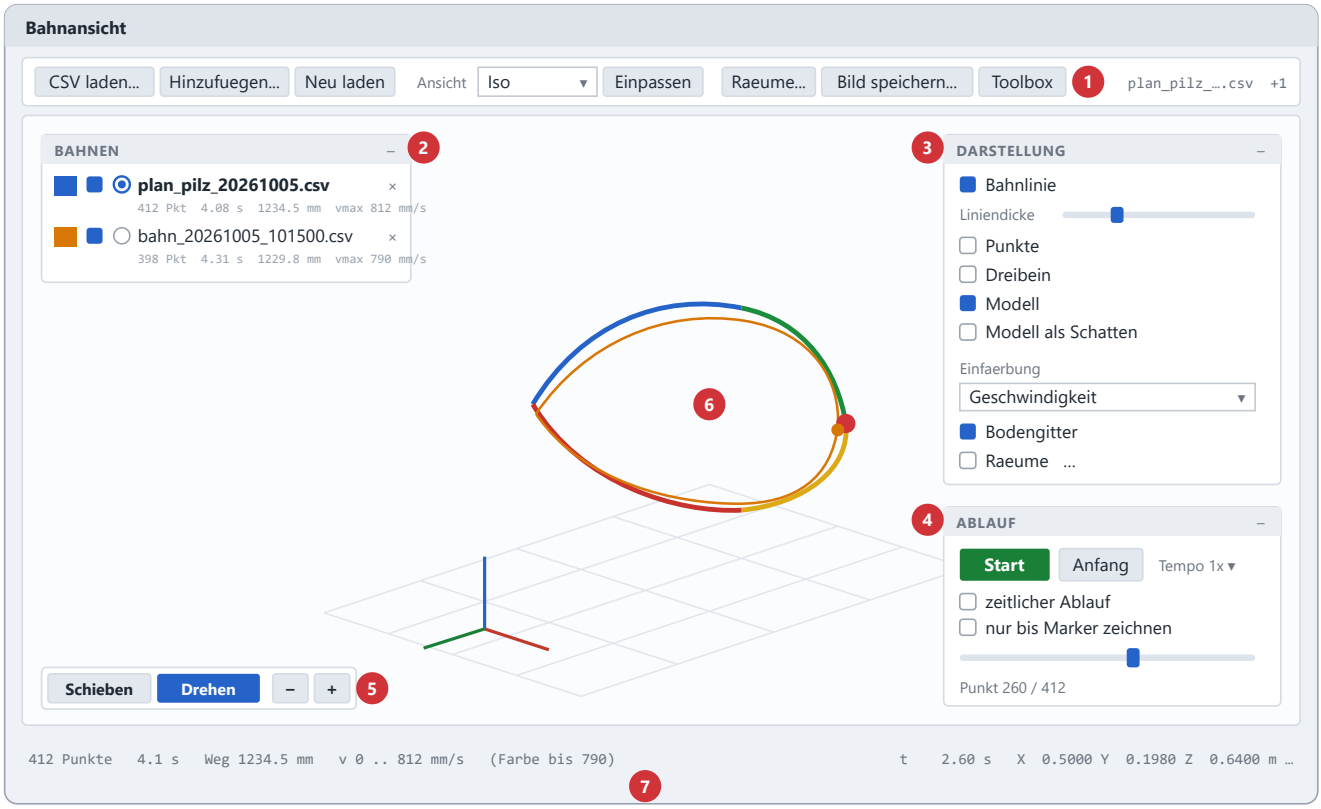
## 8.1 Was bei jeder Änderung passiert

- Jede Änderung wird vor dem Übernehmen geprüft (Namen, ein Root-Link, keine Kreise, gültige Grenzen und Achsen, Kopplungen). Ist etwas ungültig, erscheint ein Dialog und das Modell bleibt unverändert.
- Übernommene Änderungen sind sofort in RViz sichtbar, aber **noch nicht gespeichert** (Fenstertitel mit `*`).
- Ein laufender `move_group` gilt danach als **veraltet**: vor dem nächsten Planen im Parameterfenster **Neu starten**.
- **Speichern** überschreibt die Datei; die vorherige Fassung wird mit Zeitstempel in `URDF_Speicher/` gesichert. **Speichern unter…** legt eine neue Datei an, an der danach weitergearbeitet wird.
- Während einer Fahrt oder Planung sind alle Modelländerungen gesperrt.

## 8.2 Unterfenster

| Fenster | Felder | Hinweis |
|---|---|---|
| **Glied anhaengen** | neuer Link, „haengt an“, Gelenkname, Typ, Länge (Standard Z = 0,1 m), Winkel, Achse (0 0 1), Auslenkung (−180 … 180) | neuer Link ohne Masse und Geometrie |
| **Glied einfuegen** | wie oben, Vorgabe überall 0; Haken „Lage von … und allem dahinter beibehalten“ | setzt ein Glied vor das gewählte Gelenk; zeigt die Kette vorher/nachher |
| **Glied herausnehmen** | Haken „Lage der nachfolgenden Glieder beibehalten“ | nachfolgende Gelenke wandern eine Stufe hoch; bei beweglichem Gelenk fällt ein Freiheitsgrad weg |
| **Frames und Werkzeuge** | Rolle „Werkzeug (TCP)“ oder „Bezug (Basis)“, Framename, Gelenkname, „haengt an“, Länge, Winkel; **AKTIVER TCP** mit „Setzen“; Liste der festen Frames | ein Werkzeug wird als festes Gelenk unten an die Kette gehängt, ein Bezug als neuer Root-Link darüber. Der aktive TCP gilt für Anzeige und Aufzeichnung sofort, für die Planung nach Neustart von `move_group`; eine laufende Aufzeichnung wird verworfen. |
| **Reiter „Koerper“** | Link, Form (box, cylinder, sphere, mesh), Name, Größe, Ursprung (mit „Z = halbe Hoehe“), Winkel, Farbe („Waehlen…“), Deckung; Haken „Kollisionskoerper mitziehen“ | ändert `<visual>` (und wahlweise `<collision>`) |

# 9 Bahnansicht (`bahn_viewer3.py`)




*Bahnansicht (schematisch)*

Die Bahnansicht ist ein eigenes Programm. Sie braucht kein ROS und keine URDF und läuft auch neben der Oberfläche oder auf einem anderen Rechner.

```
python3 bahn_viewer3.py                                  # leer starten
python3 bahn_viewer3.py TCP_Fahrten/bahn_20261005_101500.csv
python3 bahn_viewer3.py plan_pilz_….csv plan_totg_….csv   # vergleichen
python3 bahn_viewer3.py --pruefen TCP_Fahrten/*.csv       # nur einlesen
```

1. **Werkzeugleiste** – „CSV laden…“ (ersetzt), „Hinzufuegen…“ (weitere Bahn), „Neu laden“, „Ansicht“ (Iso, XY Draufsicht, XZ vorne, YZ seitlich), „Einpassen“, „Raeume…“, „Bild speichern…“ (JPEG/PNG mit Pillow, sonst PostScript), „Toolbox“ (Fenster ein-/ausblenden).
2. **BAHNEN** – je geladene Bahn: Farbfeld (Klick wählt die Farbe), Sichtbarkeit, Auswahl der aktiven Bahn, Entfernen (×) und Kennwerte (Punkte, Dauer, Weg, Höchstgeschwindigkeit).
3. **DARSTELLUNG** – Bahnlinie und -dicke, Punkte, Dreibeine (jeder n-te Punkt), Modell (alle Frames entlang der Kette, wenn die CSV sie enthält), Dreibein je Frame, Modell aller Bahnen, Modell als Schatten, **Einfaerbung** (einfarbig, Geschwindigkeit, Zeit, Höhe Z – über alle sichtbaren Bahnen gleich skaliert), Bahnnamen, Bodengitter, Räume.
4. **ABLAUF** – „Start“/„Pause“, „Anfang“, Tempo (0,25× … 10×), „zeitlicher Ablauf“, „nur bis Marker zeichnen“ und der Slider für den Marker. Die Marker der anderen Bahnen laufen zur gleichen Zeit seit Bahnbeginn mit – so lassen sich zwei Verfahren direkt vergleichen.
5. **Schieben / Drehen, – / +** – Modus der linken Maustaste und Zoom.
6. **3D-Szene** – roter Marker = aktuelle Stelle der aktiven Bahn.
7. **Fußzeile** – Kennwerte der aktiven Bahn und Werte am Marker (Zeit, Lage, Winkel).

| Bedienung | Wirkung |
|---|---|
| linke Maustaste ziehen | drehen oder schieben (je nach Modus) |
| rechte/mittlere Maustaste ziehen | immer schieben |
| Mausrad | zoomen am Zeiger |
| Leertaste | Ablauf starten/anhalten |
| ← / → | Marker einen Punkt zurück/vor |
| F | einpassen |

**Räume** sind Quader (X/Y/Z min/max), etwa Arbeitsraum oder Hindernisse: „Hinzufuegen“, „Aendern“, „Loeschen“, „Sichtbar“, „Aus Bahn“ (Quader um die aktive Bahn), Laden/Speichern als JSON; Doppelklick schaltet die Sichtbarkeit.

## 10 Sperren und Sicherheit im Programm

- **STOP** sperrt alle Sollwerte und bremst; „FREI“ hebt das auf.
- Während `IK`, `PLANEN`, `FOLGE IK`, `FOLGE`, `FAHREN` und `BREMSEN` sind Modell, Parameter, Sollwerte und Zielfelder gesperrt; die Fußzeile zeigt den Zustand. Gesperrte Tasten sind grau.
- Angehalten wird immer über eine Rampe von 0,4 s auf der Bahn, nie durch einen Sprung.
- Gefahren wird eine Bahn nur aus dem Achsstand, für den sie gerechnet wurde.
- Planen braucht einen eigenen, bereiten und nicht veralteten `move_group`.

## 11 Meldungen und Fehlerbehebung

| Meldung (Auszug) | Ursache | Abhilfe |
|---|---|---|
| „move_group antwortet nicht. Im Parameterfenster starten …“ | `move_group` nicht gestartet | Parameter… → **Neu starten** |
| „move_group liest ein anderes Modell als dieses. Erst neu starten.“ / Fußzeile „laeuft, aber veraltet“ | Modell, TCP oder neustartpflichtige Parameter geändert | **Neu starten** |
| „move_group meldet sich seit 25 s nicht …“ | Hochlauf gescheitert (Planning Group passt nicht zur SRDF, `/joint_states` fehlt …) | letzte Zeilen von `move_group` im Meldungsfenster lesen |
| „N move_group laufen bereits (PID …)“ | Reste eines früheren Programmlaufs bieten dieselben Services an | **Fremde beenden** |
| „Keine IK-Loesung …“ + „Gefragt war …“ | Lage außerhalb des Arbeitsraums, Drehung nicht erreichbar, oder Zeitgrenze zu knapp | Lage prüfen; Kinematik → Zeitgrenze erhöhen |
| „Kein eindeutiger Endeffektor“ / „Mehrere Endeffektoren, keiner gewaehlt“ | Modell hat mehrere Leaf-Links (Endeffektoren) | Frames und Werkzeuge → **AKTIVER TCP** setzen (oder Kinematik → Tip-Link), dann Neu starten |
| „Plan verworfen: …“ / „Achsstand hat sich seit dem Planen geaendert“ | nach dem Planen bewegt oder Parameter geändert | neu planen |
| „/plan_sequence_path fehlt …“ | `move_group` ohne die Parameter dieser Oberfläche gestartet | aus dem Parameterfenster neu starten |

| Meldung (Auszug) | Ursache | Abhilfe |
|---|---|---|
| Warnung „Ruckig-Glaettung ist eingeschaltet und es stehen Radien ueber null …“ | Ruckig und Überschleifen vertragen sich nicht | Pipeline → Ruckig aus, Neu starten |
| „… Ueberschleifkugeln ueberlappen …“ | Radien zu groß für den Punktabstand | Radien verkleinern (Bericht zeigt den freien Platz) |
| „letzter Punkt traegt Radius …“ | letzte Zeile mit Radius > 0 | `radius=0` in die letzte Zeile |
| „TOTG kennt einen Override fuer die ganze Bahn …“ (ebenso Intrinsic) | `tempo` / `beschleunigung` wechseln in der Liste | nur einmal am Anfang setzen |
| „Intrinsic verrundet die ganze Kurve mit einem Wert …“ | verschiedene Radien > 0 zwischen LIN-Zeilen | einen gemeinsamen Radius verwenden |
| „TOTG/Intrinsic braucht die Parameterdatei des laufenden move_group …“ | `move_group` fehlt oder ist veraltet | Neu starten |
| „Intrinsic-Wrapper nicht ausfuehrbar …“ | Wrapper nicht gebaut oder Pfad falsch | bauen bzw. `intrinsic_wrapper:=` beim Start angeben |
| „Halte-Taster nimmt nicht an: es wird noch gebremst …“ | Bremsrampe läuft | kurz warten, neu drücken |
| „Drehkonvention nicht umgeschaltet: eine Aufzeichnung laeuft“ | Aufzeichnung aktiv | Aufzeichnung anhalten |
| „Keine TF-Ausgabe seit … s“ | TF angehalten oder ROS-Takt gestört | Haken „TF senden“ prüfen; sonst Programm neu starten |
| Fehlercode −10 „Startzustand in Kollision“ | grobe Ersatzkörper berühren sich | Pipeline → Kollisionsprüfung aus (bewusst), Neu starten |

## 11.1 Fehlercodes von MoveIt

| Code | Text in der Oberfläche | typische Ursache |
|---|---|---|
| 1 | Erfolg | – |
| 0 | Keine Antwort | Auftrag kam bei keinem Planner an (Pipeline, Pilz installiert?) |
| 99999 | Fehlgeschlagen | Sammelfehler eines Planners; Grund nur in der Ausgabe von `move_group` (wird automatisch angezeigt) |
| −2 | Planung fehlgeschlagen | Ziel erreichbar? Achsgrenzen zu eng? |
| −7 | Zeitgrenze überschritten | Planungszeit erhöhen |
| −10 | Startzustand in Kollision | SRDF-Paare / Kollisionsprüfung |
| −12 | Ziel in Kollision | – |
| −14 | Zielbedingungen verletzt | Zieltoleranz prüfen |
| −15 | Gruppenname ungültig | Planning Group passt nicht zur SRDF |
| −16 | Zielbedingungen ungültig | Gelenknamen passen nicht zur Gruppe |
| −17 / −23 | Roboterzustand ungültig / veraltet | `/joint_states` fehlt oder zu langsam |
| −31 | Keine IK-Lösung | Lage außerhalb des Arbeitsraums |

# 12 Dateien und Ablageorte

Den Aufbau des Projektordners zeigt Abschnitt 2.6.

| Ort | Inhalt |
|---|---|
| `bmKinematic.urdf` | Standardmodell des Launch-Skripts |
| `URDF_Speicher/` | automatische Sicherungen überschriebener URDF-Stände |
| `TCP_Fahrten/bahn_*.csv` | aufgezeichnete Fahrten |
| `TCP_Fahrten/plan_*.csv` | geplante Bahnen aus dem Folgenfenster |
| `/tmp/urdf_gui_moveit_*/move_group.log` | Ausgabe von `move_group` (für die Fehlersuche) |
| `/tmp/urdf_gui_totg_*/totg.log` | Ausgabe des letzten TOTG-/Intrinsic-Rechners |
| `archiv/` | frühere Programmfassungen; für den Betrieb nicht nötig |

Wer Modell, Sicherungen oder Bahnen woanders haben möchte, setzt vor dem Start eine Umgebungsvariable, z. B. `export URDF_GUI_BAHNEN=~/messungen`:

| Variable | Standard |
|---|---|
| `URDF_GUI_URDF` | `bmKinematic.urdf` neben den Programmdateien |
| `URDF_GUI_SICHERUNG` | `URDF_Speicher/` |
| `URDF_GUI_BAHNEN` | `TCP_Fahrten/` |

# 13 Fehlende Funktionen und Ausblick

## 13.1 Was die Software heute nicht kann

| Bereich | Einschränkung | Ausweg |
|---|---|---|
| Roboter | Gefahren wird nur virtuell in RViz2. Ein echter Roboter oder ein Controller (ros2_control) wird nicht angesteuert. | – |
| Fahrart CIRC | In der Punktliste fehlt ein Feld für den Hilfspunkt, im Fahrfenster ein Zwischenpunkt; CIRC scheitert deshalb immer. | Kreisbogen aus LIN-Punkten mit Überschleifen annähern |
| Reiter „Kartesisch“ | „Go“ prüft nur die Eingabe und bewegt nichts. | „Ins Fahrfenster“ |
| Punktliste | kein Laden und Speichern als Datei; beim Schließen des Fensters ist die Liste weg, beim nächsten Öffnen steht wieder das Beispiel. | Text über die Zwischenablage in eine Textdatei sichern |
| Umgebung | Hindernisse (Tisch, Maschine, Zaun) kennt die Planung nicht; geprüft wird nur der Roboter gegen sich selbst. Die „Räume“ der Bahnansicht dienen nur der Darstellung. | Bahn in der Bahnansicht mit Räumen vergleichen |
| Modell | eine Planning Group (`manipulator`) als Kette; die SRDF wird immer neu erzeugt, eine eigene SRDF lässt sich nicht laden. | – |
| TOTG | LIN wirkt nicht (Gerade im Joint Space), ein Override für die ganze Folge, der Radius der Punktliste wird nicht gelesen. | Pilz oder Intrinsic für kartesische Bahnen |
| Intrinsic | kein Jerk-Limit, keine Kollisionsprüfung, ein Verrundungsradius für alle LIN-Ecken, keine Grenze für die Drehgeschwindigkeit des TCP | Jerk und Kollision mit Pilz gegenprüfen |

## 13.2 Ausblick

Die folgende Einteilung ist ein Vorschlag aus der Durchsicht des Codes (Oktober 2026), keine Zusage. Jede Stufe bekäme eigene Tests ohne ROS, nach dem Vorbild der vorhandenen.

| Fassung (Vorschlag) | Inhalt | Aufwand |
|---|---|---|
| 2.4 | CIRC in der Punktliste mit einem Feld `hilf=X,Y,Z` für den Hilfspunkt; im Fahrfenster nur noch PTP und LIN; Taste „Go“ streichen, „Ins Fahrfenster“ wird die Haupttaste; Punktlisten laden und speichern; Quellen des Wrappers und Pilz-Patch ins Repository | klein bis mittel |
| 2.5 | Intrinsic messen: fertige Bahn auf Kollision prüfen und das Ergebnis in Bericht und CSV schreiben; Jerk je Achse ausrechnen und gegen die Joint Limits melden; Verrundung je Ecke statt eines Radius für alle | mittel |
| 3.0 | Intrinsic begrenzen: Grenze für die Drehgeschwindigkeit des TCP (Änderung im Wrapper, neuer Wrapper- und Auftragsstand); Variante „Intrinsic + Ruckig“ mit Jerk-Glättung | hoch |
| offen | Ansteuerung eines echten Roboters über ros2_control; Hindernisse in der Planning Scene; einheitliche Namen im Code (Teil I, Abschnitt 1.2) | – |